Rev_n_Roll – Közösségi platform autós események szervezésére

Szakdolgozat

Pannon Egyetem
Műszaki Informatikai Kar
Rendszer- és Számítástudományi Tanszék
Programtervező informatikus BSc

Készítette: [Név]
Neptun kód: [NEPTUNKOD]
Témavezető: [Témavezető neve]
Konzulens: [Konzulens neve]

[Évszám]

1. Bevezetés

1.1. Motiváció és problémafelvetés

Az autórajongók közössége világszerte folyamatosan növekszik, és egyre nagyobb igény mutatkozik olyan platformokra, amelyek megkönnyítik az autós események szervezését és a hasonló érdeklődésű emberek összehozását. Jelenleg a legtöbb autós esemény szervezése széttagolt módon történik: különböző közösségi média platformokon, fórumokon vagy zárt csoportokban. Ez a megközelítés számos problémát vet fel.

Elsősorban, az információk szétszórtak és nehezen hozzáférhetők. Egy adott városban vagy régióban zajló autós találkozók felkutatása időigényes folyamat, amely gyakran több platform böngészését igényli. Másodsorban, az események szervezése során hiányzik a strukturált megközelítés: nincs egységes rendszer a résztvevők kezelésére, a helyszín megosztására vagy az esemény típusának kategorizálására.

Harmadsorban, a közösségépítés nehézkes. Az autórajongók gyakran szeretnének csapatokat (crew-kat) alkotni, közös kirándulásokat szervezni vagy versenyeken részt venni, de nincs megfelelő eszköz, amely ezt támogatná. A meglévő általános közösségi platformok nem rendelkeznek az autós közösség specifikus igényeire szabott funkciókkal.

Negyedsorban, a magánjellegű és nyilvános események kezelése problematikus. Egyes szervezők szeretnék korlátozni a résztvevők körét, míg mások nyilvános eseményeket hirdetnek. A jelenlegi megoldások nem teszik lehetővé ennek rugalmas kezelését.

Végül, a földrajzi szűrés és a helyszín-alapú keresés hiánya megnehezíti az utazók számára, hogy új városokban vagy országokban autós eseményeket találjanak. Egy olyan rendszer, amely lehetővé teszi az események helyszín szerinti szűrését és a koordináták alapú keresést, jelentősen megkönnyítené az autórajongók életét.

1.2. A dolgozat célja és célkitűzései

Jelen szakdolgozat célja egy modern, webalapú közösségi platform megtervezése és implementálása, amely kifejezetten az autós események szervezésére és az autórajongók közösségének támogatására szolgál. A Rev_n_Roll elnevezésű alkalmazás átfogó megoldást kínál a fent említett problémákra.

A dolgozat fő célkitűzései a következők:

Technológiai célkitűzések:

- Modern, háromrétegű webes architektúra kialakítása, amely biztosítja a skálázhatóságot és a karbantarthatóságot.
- RESTful API tervezése és implementálása .NET 9 és ASP.NET Core technológiák felhasználásával.
- Reaktív, felhasználóbarát frontend fejlesztése Angular 19 keretrendszerrel és Material Design komponensekkel.
- Relációs adatbázis tervezése és implementálása Entity Framework Core ORM használatával.
- Biztonságos autentikációs és autorizációs rendszer kialakítása JWT token alapú megoldással.

Funkcionális célkitűzések:

- Felhasználói regisztráció és profil kezelés megvalósítása, amely lehetővé teszi az autórajongók számára saját járműveik nyilvántartását.
- Autós találkozók (meets) létrehozásának, szerkesztésének és keresésének implementálása, különböző szűrési lehetőségekkel (helyszín, címkék, dátum).
- Versenyek (races) szervezésének támogatása, különböző versenytípusok kezelésével.
- Csapatok (crews) létrehozásának és kezelésének megvalósítása, hierarchikus jogosultsági rendszerrel (vezető, társvezető, toborzó).
- Nyilvános és privát események kezelése, rugalmas láthatósági beállításokkal.
- Földrajzi koordináták alapú keresés és szűrés implementálása.

Minőségbiztosítási célkitűzések:

- Átfogó tesztelési stratégia kidolgozása és implementálása xUnit keretrendszer használatával.
- Egységtesztek (unit tests) írása az üzleti logika réteghez.
- Integrációs tesztek készítése az API végpontok validálására.
- Automatizált tesztelés beépítése a fejlesztési folyamatba.

DevOps célkitűzések:

- CI/CD pipeline kialakítása GitHub Actions segítségével.
- Automatizált build és tesztelési folyamat implementálása.
- Strukturált naplózás (logging) megvalósítása Serilog és AWS CloudWatch integrációval.

1.3. A megoldás rövid bemutatása

A Rev_n_Roll egy teljes körű webes alkalmazás, amely modern technológiai stack-re épül. A rendszer három fő rétegből áll: backend API, frontend kliens és adatbázis réteg.

Backend architektúra:

A backend .NET 9 platformon készült, ASP.NET Core keretrendszert használva. A kód Clean Architecture elvek szerint strukturált, négy fő projektre osztva:

- **ThesisBackend.Domain**: Tartalmazza a domain modelleket (User, Car, Crew, Meet, Race) és az üzeneteket (Request/Response DTO-k). Ez a réteg független minden külső függőségtől.
- **ThesisBackend.Data**: Az adatelérési réteg, amely tartalmazza a `dbContext.cs` fájlt. Ez a réteg felelős az Entity Framework Core konfigurációért, az entitások közötti kapcsolatok definiálásáért és az adatbázis migrációk kezeléséért.
- **ThesisBackend.Services**: Az üzleti logika réteg, amely service osztályokat és validátorokat tartalmaz. Itt található például az `AuthService`, amely a felhasználói regisztrációt és bejelentkezést kezeli, valamint a `TokenGenerator`, amely JWT tokenek generálásáért felelős.
- **ThesisBackend**: Az API réteg, amely a HTTP végpontokat definiálja. A controllerek (például `AuthController.cs`, `CarController.cs`, `CrewController.cs`, `MeetController.cs`, `RaceController.cs`, `UserController.cs`) fogadják a HTTP kéréseket, validálják azokat FluentValidation segítségével, és delegálják a feldolgozást a service rétegnek.

A backend biztonsági mechanizmusai közé tartozik a JWT alapú autentikáció, amely HttpOnly cookie-kban tárolja a tokeneket, valamint a jelszavak BCrypt algoritmussal történő hashelése. A `Program.cs` fájl konfigurálja a teljes alkalmazást, beleértve a dependency injection konténert, a middleware pipeline-t, a CORS beállításokat és a Serilog naplózást.

Frontend architektúra:

A frontend Angular 19 keretrendszerrel készült, TypeScript nyelven. A projekt komponens-alapú architektúrát követ, Material Design komponenseket használva a felhasználói felület kialakításához.

A főbb komponensek közé tartoznak:

- **Login és Register komponensek**: Felhasználói autentikáció kezelése
- **Profile komponens**: Felhasználói profil megjelenítése és szerkesztése, autók kezelése
- **Meets komponens**: Találkozók listázása, létrehozása, szerkesztése és szűrése
- **Races komponens**: Versenyek kezelése hasonló funkcionalitással
- **Crews komponens**: Csapatok létrehozása és kezelése

A szolgáltatások (services) réteg felelős a backend API-val való kommunikációért. Az `auth.service.ts` kezeli a bejelentkezést és a token tárolást, míg a többi service (`user.service.ts`, `car.service.ts`, `meet.service.ts`, `race.service.ts`, `crew.service.ts`) a megfelelő entitások CRUD műveleteit valósítja meg.

Az `auth.guard.ts` biztosítja, hogy csak bejelentkezett felhasználók férhessenek hozzá a védett útvonalakhoz.

Adatbázis réteg:

Az alkalmazás PostgreSQL relációs adatbázist használ. Az adatmodell öt fő entitást tartalmaz:

1. **User**: Felhasználók adatai (email, nickname, jelszó hash, leírás, profilkép)
2. **Car**: Autók adatai (márka, modell, motor, teljesítmény, leírás)
3. **Crew**: Csapatok adatai (név, leírás, kép)

4. **Meet**: Találkozók adatai (név, leírás, helyszín, koordináták, dátum, láthatóság, címkék)
5. **Race**: Versenyek adatai (név, leírás, típus, helyszín, koordináták, dátum, láthatóság)

Az entitások között komplex kapcsolatok állnak fenn:

- User és Car között 1:N kapcsolat
- User és Crew között N:M kapcsolat (UserCrew kapcsolótáblán keresztül, rangokkal)
- User és Meet között N:M kapcsolat (résztvevők)
- User és Race között N:M kapcsolat (résztvevők)
- User és Meet között 1:N kapcsolat (létrehozó)
- User és Race között 1:N kapcsolat (létrehozó)
- Crew és Meet között 1:N kapcsolat

Ezeket a kapcsolatokat a `dbContext.cs` fájlban definiált Fluent API konfiguráció kezeli.

Főbb funkciók:

1. **Felhasználói regisztráció és autentikáció**: Biztonságos regisztráció email és jelszó alapú bejelentkezéssel, JWT token alapú session kezeléssel.

2. **Autók kezelése**: Felhasználók több autót is hozzáadhatnak profiljukhoz, megadva a márka, modell, motor és teljesítmény adatokat.

3. **Találkozók szervezése**: Nyilvános vagy privát találkozók létrehozása, helyszín és koordináták megadásával, címkékkel kategorizálva (Cars N Coffee, Cruising, Meet N Greet, Amps N Woofers, Racing, Tour).

4. **Versenyek szervezése**: Különböző típusú versenyek (drag, circuit, drift, rally) létrehozása és kezelése.

5. **Csapatok (crews) kezelése**: Csapatok létrehozása, tagok meghívása különböző rangokkal (Leader, Co-Leader, Recruiter, Member).

6. **Keresés és szűrés**: Események keresése helyszín, koordináták és egyéb paraméterek alapján.

7. **Eseményekhez csatlakozás**: Felhasználók jelentkezhetnek találkozókra és versenyekre.

1.4. A dolgozat felépítése

A szakdolgozat a következő fejezetekből áll:

2. fejezet - Technológiai áttekintés: Részletes bemutatja a felhasznált technológiákat, keretrendszereket és eszközöket. Ismerteti a .NET 9, ASP.NET Core, Entity Framework Core, Angular 19, TypeScript és Material Design főbb jellemzőit, valamint indokolja a technológiai választásokat.

3. fejezet - Rendszerterv és architektúra: Bemutatja a követelményanalízis eredményeit, a funkcionális és nem-funkcionális követelményeket. Részletezi a rendszer háromrétegű architektúráját, a RESTful API tervezési elveket, valamint a használt tervezési mintákat (Repository pattern, Dependency Injection, Service layer pattern). Use case diagramok és komponens diagramok segítségével vizualizálja a rendszer működését.

4. fejezet - Adatbázis tervezés: Ismerteti az adatmodell tervezésének folyamatát, az entitások azonosítását és a kapcsolatok definiálását. Részletesen bemutatja az adatbázis sémát, az Entity Framework Core konfigurációt és a migrációk kezelését. ER diagram segítségével ábrázolja az adatbázis struktúrát.

5. fejezet - Backend implementáció: Részletesen tárgyalja a backend implementációját, a projekt struktúrát, az autentikációs és autorizációs mechanizmusokat, a validációt, az API végpontokat, a service réteget, valamint a hibakezelést és naplózást. Konkrét kódrészletekre hivatkozva mutatja be a megvalósítást.

6. fejezet - Frontend implementáció: Bemutatja az Angular projekt struktúráját, a komponensek szervezését, a szolgáltatásokat, a modelleket és a routing mechanizmust. Részletezi az autentikáció kliens oldali kezelését, a főbb komponenseket, a HTTP szolgáltatásokat és a Material Design integrációt.

7. fejezet - Tesztelés és minőségbiztosítás: Ismerteti a tesztelési stratégiát, a backend egységteszteket és integrációs teszteket. Bemutatja az xUnit keretrendszer használatát, a test coverage eredményeket és a kódminőség biztosításának módszereit.

8. fejezet - CI/CD és DevOps: Tárgyalja a GitHub Actions workflow-t, az automatizált build és tesztelési folyamatot, a deployment stratégiát és a környezeti változók kezelését.

9. fejezet - Összefoglalás és továbbfejlesztési lehetőségek: Összegzi az elért eredményeket, a tapasztalatokat, valamint felvázolja a lehetséges továbbfejlesztési irányokat (valós idejű értesítések, képfeltöltés, email értesítések, térkép integráció, mobil alkalmazás).

A dolgozat mellékletei tartalmazzák az irodalomjegyzéket, a projekt mappaszerkezetét, az API dokumentációt, az adatbázis séma diagramot és képernyőképeket az alkalmazásról.

---

2. Technológiai Áttekintés

A Rev_n_Roll platform fejlesztése során modern, ipari szabványnak számító technológiákat és keretrendszereket alkalmaztunk. Ebben a fejezetben részletesen bemutatjuk a felhasznált technológiákat, azok főbb jellemzőit, valamint indokoljuk a technológiai választásainkat.

2.1. Backend technológiák

2.1.1. .NET 9 és ASP.NET Core

A backend fejlesztéséhez a Microsoft .NET 9 platformját választottuk, amely 2024 novemberében jelent meg és a .NET ökoszisztéma legújabb Long-Term Support (LTS) verziója. A .NET 9 számos előnnyel rendelkezik, amelyek különösen alkalmassá teszik webes API-k fejlesztésére:

Teljesítmény és optimalizáció: A .NET 9 jelentős teljesítményjavításokat tartalmaz az előző verziókhoz képest. A Just-In-Time (JIT) fordító továbbfejlesztett optimalizációi, a garbage collector (GC) finomhangolása és a runtime fejlesztések együttesen akár 20-30%-os teljesítménynövekedést eredményezhetnek bizonyos workload-ok esetén. Ez különösen fontos egy közösségi platform esetében, ahol a válaszidő kritikus a felhasználói élmény szempontjából.

Cross-platform támogatás: A .NET 9 natívan támogatja a Windows, Linux és macOS operációs rendszereket. Ez rugalmasságot biztosít a deployment során, lehetővé téve, hogy az alkalmazást különböző környezetekben futtassuk anélkül, hogy a kódot módosítanunk kellene. A fejlesztés során ezt kihasználva Windows környezetben dolgoztunk, míg a production környezet Linux alapú lehet.

ASP.NET Core keretrendszer: Az ASP.NET Core a .NET platform webes alkalmazások fejlesztésére specializált keretrendszere. Főbb jellemzői:

- **Moduláris architektúra:** A middleware pipeline koncepció lehetővé teszi, hogy csak azokat a komponenseket használjuk, amelyekre szükségünk van, csökkentve ezzel az alkalmazás méretét és növelve a teljesítményt.
- **Dependency Injection (DI):** Beépített IoC (Inversion of Control) konténer, amely megkönnyíti a loosely coupled architektúra kialakítását és a tesztelhetőséget.
- **Middleware pipeline:** A HTTP kérések feldolgozása egy jól definiált pipeline-on keresztül történik, ahol minden middleware egy specifikus feladatot lát el (autentikáció, autorizáció, hibakezelés, naplózás stb.).

A `Program.cs` fájlban konfiguráljuk az ASP.NET Core alkalmazást, beleértve a szolgáltatások regisztrálását, a middleware pipeline felépítését és az alkalmazás indítását.

Miért választottuk a .NET 9-et?

1. **Érettség és stabilitás:** A .NET ökoszisztéma több mint két évtizedes fejlesztés eredménye, ami garantálja a stabilitást és a megbízhatóságot.
2. **Gazdag ökoszisztéma:** Hatalmas mennyiségű NuGet csomag áll rendelkezésre, amelyek megkönnyítik a fejlesztést (Entity Framework Core, FluentValidation, Serilog, stb.).
3. **Kiváló fejlesztői eszközök:** A Visual Studio és a Visual Studio Code kiváló támogatást nyújtanak a .NET fejlesztéshez, beleértve az IntelliSense-t, a debuggolást és a refactoring eszközöket.
4. **Teljesítmény:** A .NET 9 az egyik leggyorsabb webes keretrendszer a TechEmpower benchmarkok szerint, ami kritikus egy skálázható alkalmazás esetében.
5. **Long-Term Support:** Az LTS verzió 3 éves támogatást garantál, ami fontos a production alkalmazások számára.

2.1.2. Entity Framework Core

Az Entity Framework Core (EF Core) egy modern Object-Relational Mapping (ORM) keretrendszer, amely lehetővé teszi, hogy .NET objektumokkal dolgozzunk az adatbázis helyett, miközben az ORM automatikusan kezeli az SQL lekérdezések generálását és az adatok leképezését.

ORM koncepció előnyei:

Az ORM használata számos előnnyel jár a hagyományos SQL alapú adateléréshez képest:

- **Típusbiztonság:** A LINQ (Language Integrated Query) lekérdezések fordítási időben ellenőrizhetők, csökkentve a futásidejű hibák lehetőségét.
- **Produktivitás:** Nem kell manuálisan SQL lekérdezéseket írni és az eredményeket objektumokká konvertálni, az EF Core ezt automatikusan elvégzi.
- **Adatbázis-függetlenség:** Az EF Core támogatja a különböző adatbázis-kezelő rendszereket (PostgreSQL, SQL Server, MySQL, SQLite), így szükség esetén könnyen válthatunk közöttük.

- **Migrációk:** Az adatbázis séma változásait kód formájában követhetjük, verziókezelhetjük és automatikusan alkalmazhatjuk.

Code-First megközelítés:

A Rev_n_Roll projektben a Code-First megközelítést alkalmaztuk, ami azt jelenti, hogy először a C# osztályokat (entitásokat) definiáljuk, majd az EF Core ezekből generálja az adatbázis sémát. Ez a megközelítés előnyös, mert:

- A domain modellek a kód központi részét képezik, nem az adatbázis séma.
- A verziókezelés egyszerűbb, mivel minden az adatbázis struktúrával kapcsolatos változás kódban követhető.
- A fejlesztők C# kódban dolgoznak, ami természetesebb egy .NET projektben.

DbContext implementáció:

A `dbContext.cs` fájl tartalmazza az adatbázis kontextust, amely az EF Core központi osztálya. Ez az osztály felelős:

- A DbSet-ek definiálásáért (Cars, Users, Crews, Races, Meets)
- Az entitások közötti kapcsolatok konfigurálásáért (Fluent API)
- Az adatbázis kapcsolat kezeléséért

LINQ lekérdezések:

Az EF Core lehetővé teszi, hogy LINQ kifejezésekkel kérdezzük le az adatokat, amelyeket aztán SQL lekérdezésekké fordít. Például:

```
var publicMeets = await _context.Meets
    .Where(m => !m.Private)
    .Include(m => m.Creator)
    .Include(m => m.Users)
    .OrderByDescending(m => m.Date)
    .ToListAsync();
```

Ez a LINQ lekérdezés automatikusan SQL-lé konvertálódik, és hatékonyan lekérdezi a nyilvános találkozókat a kapcsolódó adatokkal együtt.

Migrációk kezelése:

Az EF Core migrációs rendszere lehetővé teszi az adatbázis séma verziókezelését. Amikor módosítjuk az entitásokat, egy új migrációt hozunk létre:

```
dotnet ef migrations add AddMeetTagsColumn
dotnet ef database update
```

A migrációk C# kódként tárolódnak, így verziókezelhetők és megoszthatók a csapattagokkal.

2.1.3. C# nyelvi jellemzők

A C# 12 (amely a .NET 9-cel érkezik) modern, típusbiztonságos programozási nyelv, amely számos fejlett funkciót kínál:

Nullable Reference Types:

A C# 8.0-tól kezdve a nullable reference types funkció segít elkerülni a null reference exception-öket, amelyek az egyik leggyakoribb futásidejű hibák. A projektben ezt a funkciót engedélyeztük, így a fordító figyelmeztet, ha egy változó null értéket kaphat:

```
public string Email { get; set; } // Non-nullable
public string? Description { get; set; } // Nullable
```

Async/Await programozási modell:

Az aszinkron programozás kritikus a webes alkalmazások teljesítménye szempontjából. A C# async/await kulcsszavai egyszerűvé teszik az aszinkron kód írását:

```
public async Task<User> GetUserByIdAsync(int id)
{
    return await _context.Users
        .Include(u => u.Cars)
        .FirstOrDefaultAsync(u => u.Id == id);
}
```

Az aszinkron metódusok nem blokkolják a szálat az I/O műveletek (pl. adatbázis lekérdezések) során, így több kérést tudunk párhuzamosan kezelni.

LINQ kifejezések:

A Language Integrated Query (LINQ) lehetővé teszi, hogy SQL-szerű lekérdezéseket írjunk C# kódban:

```
var userCars = cars
    .Where(c => c.UserId == userId)
    .OrderBy(c => c.Make)
    .ThenBy(c => c.Model)
    .Select(c => new CarResponse(c))
    .ToList();
```

Record types:

A C# 9.0-tól elérhető record típusok ideálisak immutable DTO-k (Data Transfer Objects) létrehozására:

```
public record LoginRequest(string Email, string Password);
```

Pattern Matching:

A pattern matching lehetővé teszi a komplex feltételek egyszerű kifejezését:

```
var result = user switch
{
    null => NotFound(),
    { IsActive: false } => Unauthorized(),
    _ => Ok(new UserResponse(user))
};
```

2.2. Frontend technológiák

2.2.1. Angular 19 keretrendszer

Az Angular a Google által fejlesztett és karbantartott, TypeScript-alapú frontend keretrendszer, amely a Single Page Application (SPA) fejlesztésére specializálódott. A Rev_n_Roll projektben az Angular 19-es verzióját használtuk, amely 2024 novemberében jelent meg.

Angular architektúra:

Az Angular egy komponens-alapú keretrendszer, amely a következő fő építőelemekből áll:

- **Komponensek (Components):** Az UI építőkövei, amelyek HTML template-et, CSS stílusokat és TypeScript logikát tartalmaznak. Minden komponens egy újrafelhasználható UI elemet reprezentál.
- **Szolgáltatások (Services):** Singleton osztályok, amelyek üzleti logikát, adatelérést vagy más megosztott funkcionalitást tartalmaznak. A szolgáltatások Dependency Injection-nel injektálhatók a komponensekbe.
- **Modulok (Modules):** Az Angular 19-ben a standalone komponensek váltak alapértelmezetté, így a modulok használata opcionális. A projektünkben standalone komponenseket használunk, ami egyszerűsíti a projekt struktúrát.
- **Direktívák (Directives):** Olyan utasítások, amelyek módosítják a DOM elemek viselkedését vagy megjelenését (pl. `*ngIf`, `*ngFor`).
- **Pipe-ok:** Adatok transzformálására szolgáló függvények a template-ekben (pl. dátum formázás, szöveg nagybetűsítés).

Reaktív programozás RxJS-sel:

Az Angular szorosan integrálódik az RxJS (Reactive Extensions for JavaScript) könyvtárral, amely reaktív programozási paradigmát tesz lehetővé. Az RxJS Observable-ök segítségével aszinkron adatfolyamokat kezelhetünk:

```
this.meetService.getMeets().subscribe({
  next: (meets) => this.meets = meets,
```

```
  error: (error) => console.error('Error loading meets:', error)
});
```

Az Observable-ök előnyei:

- Több érték kezelése időben (stream)
- Operátorok gazdag készlete (map, filter, debounceTime, switchMap, stb.)
- Automatikus leiratkozás lehetősége
- Hibakezelés beépített támogatása

Dependency Injection az Angularban:

Az Angular beépített DI rendszere lehetővé teszi a szolgáltatások injektálását a komponensekbe:

```
export class ProfileComponent {
  constructor(
    private userService: UserService,
    private carService: CarService,
    private authService: AuthService
  ) {}
}
```

A szolgáltatások `providedIn: 'root'` konfigurációval singleton-ként regisztrálódnak, így az egész alkalmazásban ugyanaz a példány használódik.

Standalone komponensek (Angular 19 újdonság):

Az Angular 19-ben a standalone komponensek váltak az alapértelmezett megközelítéssé. Ez azt jelenti, hogy a komponensek önállóan importálhatják a szükséges függőségeiket, anélkül hogy NgModule-okba kellene őket szervezni:

```
@Component({
  selector: 'app-profile',
  standalone: true,
  imports: [CommonModule, MatCardModule, MatButtonModule],
  templateUrl: './profile.component.html',
  styleUrls: ['./profile.component.css']
})
export class ProfileComponent { }
```

Ez egyszerűsíti a projekt struktúrát és csökkenti a boilerplate kódot.

Miért választottuk az Angulart?

1. **Teljes körű keretrendszer:** Az Angular "batteries included" megközelítést követ, minden szükséges eszközt tartalmaz (routing, forms, HTTP client, testing).

2. **TypeScript alapú:** A típusbiztonság és a fejlett IDE támogatás növeli a produktivitást és csökkenti a hibák számát.
3. **Vállalati támogatás:** A Google fejleszti és karbantartja, ami hosszú távú stabilitást garantál.
4. **Strukturált architektúra:** Az Angular erős véleménnyel rendelkezik a projekt struktúrájáról, ami megkönnyíti a karbantartást és a csapatmunkát.
5. **Material Design integráció:** Az Angular Material könyvtár kiváló Material Design komponenseket biztosít.

A `package.json` fájl tartalmazza az Angular projekt függőségeit, beleértve az Angular 19 core csomagokat és az Angular Material komponenseket.

2.2.2. TypeScript

A TypeScript a JavaScript típusbiztonságos szupersete, amelyet a Microsoft fejlesztett. A TypeScript kód JavaScript-té fordítódik, így bármely JavaScript környezetben futtatható.

Típusbiztonság előnyei:

A TypeScript fő előnye a statikus típusrendszer, amely fordítási időben ellenőrzi a típusokat:

```
interface User {
  id: number;
  email: string;
  nickname: string;
  description?: string;
}

function getUserById(id: number): Promise<User> {
  return this.http.get<User>(`${this.apiUrl}/${id}`).toPromise();
}
```

Ha rossz típusú paramétert adunk át, a fordító hibát jelez, még azelőtt, hogy a kódot futtatnánk.

Interfészek és típusok:

A TypeScript interfészek és típusok segítségével definiálhatjuk az adatstruktúrákat:

```
export interface Car {
  id: number;
  userId: number;
  make: string;
  model: string;
  engine: string;
  power: number;
  description: string;
}
```

```
export enum MeetTags {
  CarsNCoffee = 0,
  Cruising = 1,
  MeetNGreet = 2,
  AmpsNWoofers = 3,
  Racing = 4,
  Tour = 5
}
```

Ezek az interfészek és enum-ok a `rev-n-roll/src/app/models/` mappában találhatók.

Fejlesztői élmény javítása:

A TypeScript jelentősen javítja a fejlesztői élményt:

- **IntelliSense:** Az IDE automatikus kódkiegészítést és dokumentációt nyújt.
- **Refactoring:** Biztonságos refactoring eszközök (rename, extract method, stb.).
- **Navigáció:** Könnyű navigáció a kódban (go to definition, find all references).
- **Hibák korai felismerése:** A legtöbb hiba fordítási időben kiderül, nem futásidőben.

Kód karbantarthatóság:

A típusok dokumentációként is szolgálnak, megkönnyítve a kód megértését:

```
createMeet(meetData: MeetRequest): Observable<MeetResponse> {
  return this.http.post<MeetResponse>(`${this.apiUrl}`, meetData);
}
```

Ebből a függvényből egyértelműen látszik, hogy milyen típusú adatot vár paraméterként és mit ad vissza.

2.2.3. Material Design

A Material Design a Google által kifejlesztett design nyelv, amely konzisztens és intuitív felhasználói élményt biztosít. Az Angular Material az Angular hivatalos Material Design komponens könyvtára.

Google Material Design elvek:

A Material Design három fő elvre épül:

1. **Material is the metaphor:** A fizikai világ törvényei (fény, árnyék, mozgás) inspirálják a digitális felületet.

2. **Bold, graphic, intentional:** Merész színek, tipográfia és fehér tér tudatos használata.

3. **Motion provides meaning:** Az animációk és átmenetek segítenek megérteni a felület működését.

Angular Material komponensek:

A projektben számos Angular Material komponenst használunk:

- **MatCard:** Kártyák megjelenítésére (találkozók, versenyek, autók)
- **MatButton:** Gombok különböző stílusokban

- **MatFormField, MatInput:** Form mezők
- **MatSelect:** Legördülő listák
- **MatDatepicker:** Dátumválasztó
- **MatDialog:** Modális dialógusok
- **MatChipList:** Címkék megjelenítése
- **MatIcon:** Material ikonok
- **MatToolbar:** Navigációs sáv

Példa egy Material Design dialógus használatára:

```
openAddCarDialog(): void {
  const dialogRef = this.dialog.open(AddCarDialogComponent, {
    width: '500px',
    data: { userId: this.userId }
  });

  dialogRef.afterClosed().subscribe(result => {
    if (result) {
      this.loadCars();
    }
  });
}
```

Egységes felhasználói élmény:

Az Angular Material komponensek használata biztosítja, hogy:

- Az alkalmazás minden része konzisztens megjelenésű
- A felhasználók ismerős interakciós mintákat találnak
- Az accessibility (akadálymentesség) beépített
- A reszponzív design automatikusan működik

Reszponzív design támogatás:

Az Angular Material komponensek automatikusan alkalmazkodnak a különböző képernyőméretekhez. A Flexbox Layout és a CSS Grid használatával könnyen létrehozhatunk reszponzív elrendezéseket.

2.3. Adatbázis technológia

2.3.1. PostgreSQL

A PostgreSQL egy nyílt forráskódú, objektum-relációs adatbázis-kezelő rendszer (ORDBMS), amely több mint 35 éves fejlesztés eredménye. A Rev_n_Roll projektben a PostgreSQL-t választottuk adatbázis-kezelő rendszerként.

Relációs adatbázis-kezelő rendszer:

A PostgreSQL egy teljes értékű RDBMS, amely támogatja:

- **SQL szabvány:** Szinte teljes SQL:2016 szabvány támogatás

- **Tranzakciók:** ACID (Atomicity, Consistency, Isolation, Durability) tulajdonságok
- **Referenciális integritás:** Foreign key-k, unique constraints, check constraints
- **Indexek:** B-tree, Hash, GiST, SP-GiST, GIN, BRIN indexek
- **Nézetek (Views):** Egyszerű és materializált nézetek
- **Tárolt eljárások:** Functions és stored procedures

ACID tulajdonságok:

Az ACID tulajdonságok garantálják az adatok konzisztenciáját:

- **Atomicity (Atomosság):** Egy tranzakció vagy teljesen végrehajtódik, vagy egyáltalán nem.
- **Consistency (Konzisztencia):** A tranzakció az adatbázist egy konzisztens állapotból egy másik konzisztens állapotba viszi.
- **Isolation (Izoláció):** A párhuzamosan futó tranzakciók nem zavarják egymást.
- **Durability (Tartósság):** A commitált tranzakciók változásai megmaradnak rendszerhiba esetén is.

Teljesítmény és megbízhatóság:

A PostgreSQL kiváló teljesítményt nyújt:

- **MVCC (Multi-Version Concurrency Control):** Lehetővé teszi a nagy párhuzamosságot anélkül, hogy az olvasások blokkolnák az írásokat.
- **Query optimizer:** Intelligens lekérdezés optimalizáló, amely a leghatékonyabb végrehajtási tervet választja.
- **Connection pooling:** PgBouncer vagy beépített connection pooling a kapcsolatok hatékony kezelésére.
- **Replikáció:** Streaming replication és logical replication támogatás.

JSON támogatás:

A PostgreSQL natívan támogatja a JSON és JSONB adattípusokat, ami hasznos lehet félig strukturált adatok tárolására. Bár a projektben ezt nem használjuk ki teljes mértékben, a jövőbeni bővítések (pl. esemény metaadatok) számára előnyös lehet.

Miért választottuk a PostgreSQL-t?

1. **Nyílt forráskód:** Ingyenes és szabadon használható, nincs licencdíj.

2. **Megbízhatóság:** Évtizedes fejlesztés és production használat bizonyítja a stabilitását.

3. **Funkciók gazdagsága:** Fejlett funkciók (pl. full-text search, GIS támogatás PostGIS-sel, JSON).

4. **Közösség:** Nagy és aktív közösség, rengeteg dokumentáció és támogatás.

5. **EF Core támogatás:** Kiváló Npgsql provider az Entity Framework Core-hoz.

6. **Skálázhatóság:** Képes kezelni nagy adatmennyiségeket és magas terhelést.

2.3.2. Relációs adatmodell

A relációs adatmodell az adatok táblákban történő szervezésén alapul, ahol a táblák közötti kapcsolatokat foreign key-k definiálják.

Normalizáció:

Az adatbázis tervezés során a normalizációs szabályokat követtük, hogy elkerüljük az adatredundanciát és biztosítsuk az adatok konzisztenciáját. A projektben a harmadik normálformát (3NF) alkalmaztuk:

- **1NF (First Normal Form):** Minden oszlop atomi értéket tartalmaz, nincs ismétlődő csoport.
- **2NF (Second Normal Form):** Minden nem-kulcs attribútum teljesen függ a primary key-től.
- **3NF (Third Normal Form):** Nincs tranzitív függőség, minden nem-kulcs attribútum közvetlenül függ a primary key-től.

Referenciális integritás:

A foreign key constraints biztosítják, hogy a kapcsolatok konzisztensek maradjanak:

```
ALTER TABLE "Cars"
ADD CONSTRAINT "FK_Cars_Users_UserId"
FOREIGN KEY ("UserId") REFERENCES "Users" ("Id")
ON DELETE CASCADE;
```

Ez garantálja, hogy egy autó mindig egy létező felhasználóhoz tartozik, és ha a felhasználót töröljük, az autói is törlődnek.

Indexek és megszorítások:

Az indexek gyorsítják a lekérdezéseket, míg a megszorítások biztosítják az adatok integritását:

- **Unique indexek:** Email és Nickname egyediségének biztosítása
- **Primary key indexek:** Automatikusan létrejönnek az Id mezőkön
- **Foreign key indexek:** Gyorsítják a join műveleteket

A `dbContext.cs` fájlban definiáljuk ezeket a megszorításokat a Fluent API segítségével.

2.4. Egyéb technológiák

2.4.1. JWT autentikáció

A JSON Web Token (JWT) egy nyílt szabvány (RFC 7519), amely kompakt és önálló módon továbbít információt felek között JSON objektumként. A Rev_n_Roll projektben JWT-t használunk a felhasználói autentikációhoz.

JWT működése:

Egy JWT három részből áll, amelyeket pont (.) karakter választ el:

1. **Header:** Tartalmazza a token típusát (JWT) és az aláírási algoritmust (HS256).
2. **Payload:** Tartalmazza a claims-eket (állításokat), pl. felhasználó ID, email, lejárati idő.
3. **Signature:** Az aláírás, amely biztosítja a token hitelességét.

Példa JWT struktúra:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gR
G9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Stateless autentikáció előnyei:

A JWT-alapú autentikáció stateless, ami azt jelenti, hogy a szervernek nem kell session információt tárolnia:

- **Skálázhatóság:** Nincs szükség session store-ra, így könnyebb horizontálisan skálázni.
- **Teljesítmény:** Nincs adatbázis lekérdezés minden kérésnél a session ellenőrzésére.
- **Microservices:** Könnyen megosztható több szolgáltatás között.

HttpOnly cookie-k biztonsága:

A projektben a JWT tokent HttpOnly cookie-ban tároljuk, nem localStorage-ban. Ez védelmet nyújt az XSS (Cross-Site Scripting) támadások ellen, mivel a JavaScript nem fér hozzá a cookie-hoz:

```
Response.Cookies.Append("jwt", token, new CookieOptions
{
    HttpOnly = true,
    Secure = true,
    SameSite = SameSiteMode.Strict,
    Expires = DateTime.UtcNow.AddMinutes(60)
});
```

Token generálás és validáció:

A `TokenGenerator.cs` fájl tartalmazza a JWT token generálás logikáját:

```
public string GenerateAccessToken(User user)
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var secret = Encoding.ASCII.GetBytes(_jwtSettings.Secret);
    var tokenDescriptor = new SecurityTokenDescriptor()
    {
        Subject = new ClaimsIdentity(new Claim[]
        {
            new Claim(ClaimTypes.Name, user.Id.ToString())
        }),
        Expires = DateTime.UtcNow.AddMinutes(_jwtSettings.DurationInMinutes),
        Issuer = _jwtSettings.Issuer,
        Audience = _jwtSettings.Audience,
        SigningCredentials = new SigningCredentials(
            new SymmetricSecurityKey(secret),
            SecurityAlgorithms.HmacSha256Signature)
    };
    var token = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(token);
}
```

A token validálása automatikusan történik az ASP.NET Core JWT middleware által, amelyet a `Program.cs` fájlban konfigurálunk.

2.4.2. FluentValidation

A FluentValidation egy .NET könyvtár, amely lehetővé teszi a validációs szabályok deklaratív módon történő definiálását. A projektben a FluentValidation-t használjuk a bejövő HTTP kérések validálására.

Deklaratív validáció:

A FluentValidation lehetővé teszi, hogy a validációs szabályokat külön osztályokban definiáljuk, fluent API stílusban:

```
public class RegistrationRequestValidator : AbstractValidator<RegistrationRequest>
{
    public RegistrationRequestValidator()
    {
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Invalid email format.");

        RuleFor(x => x.Nickname)
            .NotEmpty().WithMessage("Nickname is required.")
            .Length(3, 32).WithMessage("Nickname must be between 3 and 32
characters.");

        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Password is required.")
            .MinimumLength(8).WithMessage("Password must be at least 8 characters
long.");
    }
}
```

Validációs szabályok szervezése:

A validátorok külön osztályokban vannak, ami:

- Szeparálja a validációs logikát az üzleti logikától
- Újrafelhasználhatóvá teszi a validációs szabályokat
- Könnyebbé teszi a tesztelést

Hibakezelés és hibaüzenetek:

A FluentValidation automatikusan generál strukturált hibaüzeneteket, amelyeket az ASP.NET Core automatikusan ValidationProblemDetails formátumban ad vissza:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
```

```
    "errors": {
      "Email": ["Invalid email format."],
      "Password": ["Password must be at least 8 characters long."]
    }
  }
```

2.4.3. Serilog naplózás

A Serilog egy strukturált naplózó könyvtár .NET-hez, amely lehetővé teszi, hogy a log üzeneteket strukturált formában tároljuk, nem csak egyszerű szövegként.

Strukturált naplózás koncepciója:

A hagyományos naplózás szöveges üzeneteket ír ki:

```
User logged in: john@example.com at 2024-03-03 12:00:00
```

A strukturált naplózás JSON formátumban tárolja az adatokat:

```
{
  "Timestamp": "2024-03-03T12:00:00Z",
  "Level": "Information",
  "Message": "User logged in",
  "Properties": {
    "Email": "john@example.com",
    "UserId": 123
  }
}
```

Ez lehetővé teszi a logok hatékony keresését és szűrését.

Log szintek:

A Serilog különböző log szinteket támogat:

- **Verbose:** Részletes debug információk
- **Debug:** Debug információk fejlesztéshez
- **Information:** Általános információs üzenetek
- **Warning:** Figyelmeztetések
- **Error:** Hibák
- **Fatal:** Kritikus hibák

A `appsettings.json` fájlban konfiguráljuk a log szinteket:

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
```

```
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.EntityFrameworkCore.Database.Command": "Warning",
        "ThesisBackend": "Debug"
      }
    }
  }
```

Sink-ek (Console, File, CloudWatch):

A Serilog sink-ek határozzák meg, hogy a logok hova kerüljenek:

- **Console sink:** Logok megjelenítése a konzolon (fejlesztés során)
- **File sink:** Logok fájlba írása
- **CloudWatch sink:** Logok küldése AWS CloudWatch-ba (production)

A `Program.cs` fájlban konfiguráljuk a Serilog-ot:

```
builder.Host.UseSerilog((context, services, loggerConfiguration) =>
{
    loggerConfiguration
        .ReadFrom.Configuration(context.Configuration)
        .ReadFrom.Services(services)
        .Enrich.FromLogContext()
        .WriteTo.Console()
        .WriteTo.AWSSeriLog();
});
```

2.4.4. AWS CloudWatch

Az AWS CloudWatch egy monitoring és naplózó szolgáltatás, amely lehetővé teszi a központi log gyűjtést és elemzést.

Felhő-alapú naplózás:

A CloudWatch előnyei:

- **Központosított logok:** Minden alkalmazás példány ugyanoda küldi a logokat
- **Perzisztencia:** A logok megmaradnak, még ha az alkalmazás leáll is
- **Keresés és szűrés:** Hatékony log keresés és szűrés
- **Riasztások:** Automatikus riasztások bizonyos log minták esetén

Központi log gyűjtés:

A projektben a Serilog AWS CloudWatch sink-et használjuk a logok CloudWatch-ba küldésére:

```
  "CloudWatch": {
    "LogGroupName": "RevNRoll-Thesis-Logs-Dev",
```

```
  "Region": "eu-north-1"
}
```

Monitoring és riasztások:

A CloudWatch lehetővé teszi:

- **Metrics:** Metrikák gyűjtése (pl. hány hiba történt az elmúlt órában)
- **Alarms:** Riasztások beállítása (pl. email küldés, ha sok hiba van)
- **Dashboards:** Vizuális dashboardok létrehozása a metrikákból

Ez különösen hasznos production környezetben, ahol gyorsan észre kell vennünk a problémákat.

---

3. Rendszerterv és Architektúra

A Rev_n_Roll platform tervezése során kiemelt figyelmet fordítottunk a skálázhatóságra, karbantarthatóságra és a tiszta kód elvekre. Ebben a fejezetben bemutatjuk a követelményanalízis eredményeit, a rendszer architektúráját, valamint a használt tervezési mintákat.

3.1. Követelményanalízis

3.1.1. Funkcionális követelmények

A funkcionális követelmények meghatározzák, hogy a rendszer milyen funkciókat kell, hogy nyújtson a felhasználók számára. A Rev_n_Roll platform esetében a következő főbb funkcionális követelményeket azonosítottuk:

Felhasználókezelés:

- **FR-01:** A rendszernek lehetővé kell tennie új felhasználók regisztrációját email cím és jelszó megadásával.
- **FR-02:** A felhasználóknak egyedi nickname-kel kell rendelkezniük, amely azonosítja őket a platformon.
- **FR-03:** A rendszernek támogatnia kell a felhasználói bejelentkezést email és jelszó alapján.
- **FR-04:** A felhasználóknak lehetőségük kell legyen profiljuk szerkesztésére (nickname, leírás, profilkép).
- **FR-05:** A rendszernek biztonságosan kell tárolnia a jelszavakat (hash-elve, nem plain text formában).

Autókezelés:

- **FR-06:** A felhasználóknak lehetőségük kell legyen több autó hozzáadására profiljukhoz.
- **FR-07:** Minden autóhoz meg kell adni a márkát, modellt, motort, teljesítményt és opcionális leírást.
- **FR-08:** A felhasználóknak lehetőségük kell legyen autóik szerkesztésére és törlésére.
- **FR-09:** A rendszernek meg kell jelenítenie egy felhasználó összes autóját a profil oldalon.

Találkozók (Meets) kezelése:

- **FR-10:** A felhasználóknak lehetőségük kell legyen találkozók létrehozására.
- **FR-11:** Minden találkozóhoz meg kell adni: nevet, leírást, helyszínt, koordinátákat, dátumot és időpontot.

- **FR-12:** A találkozók lehetnek nyilvánosak vagy privátak.
- **FR-13:** A találkozókhoz címkéket lehet rendelni (Cars N Coffee, Cruising, Meet N Greet, Amps N Woofers, Racing, Tour).
- **FR-14:** A találkozók opcionálisan hozzárendelhetők egy crew-hoz.
- **FR-15:** A felhasználóknak lehetőségük kell legyen találkozókra jelentkezni.
- **FR-16:** A rendszernek támogatnia kell a találkozók szűrését helyszín, koordináták és címkék alapján.
- **FR-17:** A találkozók létrehozójának lehetősége kell legyen a találkozó szerkesztésére és törlésére.

Versenyek (Races) kezelése:

- **FR-18:** A felhasználóknak lehetőségük kell legyen versenyek létrehozására.
- **FR-19:** Minden versenyhez meg kell adni: nevet, leírást, típust (drag, circuit, drift, rally), helyszínt, koordinátákat, dátumot.
- **FR-20:** A versenyek lehetnek nyilvánosak vagy privátak.
- **FR-21:** A versenyek opcionálisan hozzárendelhetők egy crew-hoz.
- **FR-22:** A felhasználóknak lehetőségük kell legyen versenyekre jelentkezni.
- **FR-23:** A rendszernek támogatnia kell a versenyek szűrését helyszín, koordináták és típus alapján.
- **FR-24:** A versenyek létrehozójának lehetősége kell legyen a verseny szerkesztésére és törlésére.

Csapatok (Crews) kezelése:

- **FR-25:** A felhasználóknak lehetőségük kell legyen crew-k létrehozására.
- **FR-26:** Minden crew-hoz meg kell adni: nevet, leírást és opcionális képet.
- **FR-27:** A crew-k hierarchikus rangrendszerrel rendelkeznek: Leader, Co-Leader, Recruiter, Member.
- **FR-28:** A crew vezetőinek (Leader, Co-Leader) lehetőségük kell legyen tagok hozzáadására és eltávolítására.
- **FR-29:** A crew-k szervezhetnek találkozókat és versenyeket.
- **FR-30:** A rendszernek meg kell jelenítenie egy crew összes tagját és eseményét.

Keresés és szűrés:

- **FR-31:** A rendszernek támogatnia kell a találkozók és versenyek keresését helyszín alapján.
- **FR-32:** A rendszernek támogatnia kell a földrajzi koordináták alapú keresést (adott távolságon belüli események).
- **FR-33:** A rendszernek támogatnia kell a címkék szerinti szűrést találkozók esetében.
- **FR-34:** A rendszernek támogatnia kell a versenytípus szerinti szűrést versenyek esetében.

3.1.2. Nem-funkcionális követelmények

A nem-funkcionális követelmények a rendszer minőségi jellemzőit határozzák meg:

Teljesítmény:

- **NFR-01:** Az API végpontoknak 95%-ban 200 ms alatt kell válaszolniuk normál terhelés mellett.
- **NFR-02:** A rendszernek képesnek kell lennie legalább 100 egyidejű felhasználó kiszolgálására.
- **NFR-03:** Az adatbázis lekérdezéseknek optimalizáltnak kell lenniük (indexek használata, N+1 probléma elkerülése).

Biztonság:

- **NFR-04:** A jelszavakat BCrypt algoritmussal kell hash-elni, minimum 10 salt round-dal.
- **NFR-05:** A JWT tokeneknek HttpOnly cookie-kban kell tárolódniuk, XSS támadások ellen védve.
- **NFR-06:** A JWT tokenek élettartama maximum 60 perc lehet.
- **NFR-07:** Az API végpontoknak védettnek kell lenniük autentikáció és autorizáció ellen.
- **NFR-08:** A CORS beállításoknak korlátozniuk kell a hozzáférést csak megbízható origin-ekre.

Skálázhatóság:

- **NFR-09:** A rendszer architektúrájának támogatnia kell a horizontális skálázást.
- **NFR-10:** Az alkalmazásnak stateless-nek kell lennie (session adatok nem tárolódnak a szerverben).
- **NFR-11:** Az adatbázis kapcsolatoknak connection pooling-ot kell használniuk.

Karbantarthatóság:

- **NFR-12:** A kódnak követnie kell a SOLID elveket és a Clean Architecture mintát.
- **NFR-13:** A kód lefedettségnek (code coverage) legalább 70%-nak kell lennie unit tesztek esetében.
- **NFR-14:** A kódnak követnie kell a C# és TypeScript coding conventions-t.
- **NFR-15:** A rendszernek strukturált naplózást kell használnia (Serilog).

Használhatóság:

- **NFR-16:** A felhasználói felületnek responsívnak kell lennie (mobil, tablet, desktop).
- **NFR-17:** A felületnek követnie kell a Material Design irányelveket.
- **NFR-18:** A hibakezelésnek felhasználóbarát hibaüzeneteket kell megjelenítenie.

Rendelkezésre állás:

- **NFR-19:** A rendszernek legalább 99% uptime-ot kell biztosítania production környezetben.
- **NFR-20:** A rendszernek graceful degradation-t kell támogatnia (ha egy szolgáltatás leáll, a többi továbbra is működik).

3.2. Rendszer architektúra

3.2.1. Háromrétegű architektúra

A Rev_n_Roll platform háromrétegű (three-tier) architektúrát követ, amely elválasztja a prezentációs réteget, az üzleti logika réteget és az adatelérési réteget. Ez a szeparáció számos előnnyel jár:

Rétegek szeparációja:

1. **Prezentációs réteg (Frontend):** Angular 19 alapú SPA, amely a felhasználói felületet biztosítja. Ez a réteg felelős a felhasználói interakciók kezeléséért, az adatok megjelenítéséért és a backend API-val való kommunikációért.

2. **Üzleti logika réteg (Backend API):** ASP.NET Core alapú RESTful API, amely az üzleti logikát implementálja. Ez a réteg felelős a kérések validálásáért, az üzleti szabályok végrehajtásáért és az adatelérési réteg koordinálásáért.

3. **Adatelérési réteg (Database):** PostgreSQL relációs adatbázis, amely az alkalmazás adatait tárolja. Az Entity Framework Core ORM biztosítja az absztrakciós réteget a backend és az adatbázis között.

Előnyök:

- **Separation of Concerns:** Minden réteg egy jól definiált felelősséggel rendelkezik.
- **Skálázhatóság:** A rétegek egymástól függetlenül skálázhatók.
- **Karbantarthatóság:** A változások egy rétegben nem érintik a többi réteget.
- **Tesztelhetőség:** Minden réteg külön-külön tesztelhető.
- **Technológiai függetlenség:** A rétegek technológiája egymástól függetlenül cserélhető.

3.2.2. Clean Architecture implementáció

A backend projekt Clean Architecture (más néven Onion Architecture vagy Hexagonal Architecture) elvek szerint strukturált. Ez az architektúra a függőségek irányát szabályozza: a belső rétegek nem függhetnek a külső rétegektől.

Projekt struktúra:

A backend négy fő projektre oszlik:

1. **ThesisBackend.Domain (Core réteg):**
   - Tartalmazza a domain modelleket (`User.cs`, `Car.cs`, `Crew.cs`, `Meet.cs`, `Race.cs`)
   - Tartalmazza az üzeneteket (Request/Response DTO-k)
   - Nem függ semmilyen külső library-től vagy keretrendszertől
   - Ez a legbelső réteg, amely az üzleti entitásokat reprezentálja
2. **ThesisBackend.Data (Infrastructure réteg):**
   - Tartalmazza a `dbContext.cs` fájlt
   - Felelős az Entity Framework Core konfigurációért
   - Implementálja az adatbázis kapcsolatokat és migrációkat
   - Függ a Domain rétegtől és az EF Core-tól
3. **ThesisBackend.Services (Application réteg):**
   - Tartalmazza az üzleti logikát (service osztályok)
   - Tartalmazza a validátorokat (FluentValidation)
   - Tartalmazza az interfészeket (IAuthService, IPasswordHasher, ITokenGenerator)
   - Függ a Domain rétegtől és a Data rétegtől
   - Implementálja a use case-eket
4. **ThesisBackend (Presentation réteg):**
   - Tartalmazza a controllereket (`AuthController.cs`, `CarController.cs`, stb.)
   - Tartalmazza a `Program.cs` fájlt (alkalmazás belépési pont)
   - Felelős a HTTP kérések fogadásáért és válaszok küldéséért
   - Függ minden más rétegtől

Függőségi irányok:

```
ThesisBackend (Presentation)
    ↓
ThesisBackend.Services (Application)
    ↓
```

```
ThesisBackend.Data (Infrastructure)
    ↓
ThesisBackend.Domain (Core)
```

A Domain réteg nem függ semmitől, így könnyen tesztelhető és újrafelhasználható. A külső rétegek függnek a belső rétegektől, de fordítva nem.

3.2.3. RESTful API tervezés

A Rev_n_Roll backend egy RESTful API-t implementál, amely HTTP protokollon keresztül kommunikál a frontend-del. A RESTful API tervezése során a következő elveket követtük:

HTTP metódusok használata:

- **GET:** Adatok lekérdezése (pl. `GET /api/v1/Meet/getMeets`)
- **POST:** Új erőforrás létrehozása (pl. `POST /api/v1/Authentication/register`)
- **PUT:** Meglévő erőforrás teljes frissítése (pl. `PUT /api/v1/Meet/updateMeet/{meetId}`)
- **DELETE:** Erőforrás törlése (pl. `DELETE /api/v1/Meet/deleteMeet/{meetId}`)

Erőforrás-orientált URL-ek:

Az API végpontok erőforrásokat reprezentálnak, nem műveleteket. Az `ApiPlan.md` fájl részletesen dokumentálja az összes végpontot:

```
/api/v1/Authentication/register  - Felhasználó regisztráció
/api/v1/Authentication/login     - Felhasználó bejelentkezés
/api/v1/Meet/addMeet/{userId}    - Találkozó létrehozása
/api/v1/Meet/getMeets            - Összes találkozó lekérdezése
/api/v1/Meet/getMeetsF           - Találkozók szűrt lekérdezése
/api/v1/Race/addRace/{userId}    - Verseny létrehozása
/api/v1/Crew/addCrew/{userId}    - Crew létrehozása
```

HTTP státuszkódok:

Az API megfelelő HTTP státuszkódokat használ:

- **200 OK:** Sikeres kérés
- **201 Created:** Sikeres létrehozás
- **400 Bad Request:** Validációs hiba
- **401 Unauthorized:** Autentikáció szükséges
- **403 Forbidden:** Nincs jogosultság
- **404 Not Found:** Erőforrás nem található
- **500 Internal Server Error:** Szerver oldali hiba

JSON formátum:

Az API JSON formátumban kommunikál. Példa egy találkozó létrehozására:

```
POST /api/v1/Meet/addMeet/1
{
  "name": "Supercar Sunday",
  "description": "Biggest car meet in the city",
  "location": "Downtown Parking Lot",
  "coordinates": "40.7128,-74.0060",
  "date": "2024-06-20T15:00:00Z",
  "private": false,
  "crewId": 3,
  "tags": [0, 5]
}
```

Válasz:

```
{
  "id": 201,
  "name": "Supercar Sunday",
  "description": "Biggest car meet in the city",
  "location": "Downtown Parking Lot",
  "coordinates": "40.7128,-74.0060",
  "date": "2024-06-20T15:00:00Z",
  "private": false,
  "creatorId": 1,
  "crewId": 3,
  "tags": ["CarsNCoffee", "Tour"]
}
```

Verziókezelés:

Az API URL-ben tartalmazza a verziót (`/api/v1/`), ami lehetővé teszi a backward compatibility fenntartását jövőbeli változások esetén.

3.3. Tervezési minták

3.3.1. Repository Pattern

A Repository Pattern egy absztrakciós réteget biztosít az adatelérés és az üzleti logika között. Bár a projektben közvetlenül az Entity Framework Core DbContext-et használjuk (ami maga is egy Repository Pattern implementáció), a service réteg elkülöníti az adatelérési logikát az üzleti logikától.

Előnyök:

- **Absztrakció:** Az üzleti logika nem függ közvetlenül az adatbázistól
- **Tesztelhetőség:** A service réteg könnyen mockolható tesztelés során
- **Központosított adatelérés:** Az adatbázis lekérdezések egy helyen vannak

Implementáció példa:

A `MeetController.cs` nem közvetlenül a `dbContext`-et használja, hanem az `IMeetService` interfészen keresztül:

```
public class MeetController : ControllerBase
{
    private readonly IMeetService _meetService;

    public MeetController(IMeetService meetService)
    {
        _meetService = meetService;
    }

    [HttpGet("getMeets")]
    public async Task<ActionResult<List<SmallEventResponse>>> GetAllMeets()
    {
        var result = await _meetService.GetAllMeetsAsync();
        return result.Success ? Ok(result.Meets) : NotFound(result.ErrorMessage);
    }
}
```

3.3.2. Dependency Injection

A Dependency Injection (DI) egy tervezési minta, amely lehetővé teszi a loosely coupled architektúra kialakítását. Az ASP.NET Core beépített DI konténert használ.

Service regisztráció:

A `Program.cs` fájlban regisztráljuk a szolgáltatásokat:

```
// Authentication services
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IPasswordHasher, PasswordHasher>();
builder.Services.AddScoped<ITokenGenerator, TokenGenerator>();

// Validators
builder.Services.AddValidatorsFromAssemblyContaining<RegistrationRequestValidator>
();

// Database context
builder.Services.AddDbContext<dbContext>(options =>
    options.UseNpgsql(connectionString));
```

Service élettartamok:

- **Transient:** Minden kérésnél új példány (pl. validátorok)
- **Scoped:** HTTP kérésenként egy példány (pl. DbContext, service-ek)
- **Singleton:** Alkalmazás élettartamára egy példány (pl. konfigurációk)

Előnyök:

- **Tesztelhetőség:** Könnyen cserélhetők a függőségek mock implementációkra
- **Loosely coupled:** Az osztályok nem függenek konkrét implementációktól
- **Konfigurálhatóság:** A függőségek központilag konfigurálhatók

3.3.3. Service Layer Pattern

A Service Layer Pattern elkülöníti az üzleti logikát a prezentációs rétegtől. A service osztályok felelősek az üzleti szabályok végrehajtásáért, az adatok validálásáért és az adatelérési réteg koordinálásáért.

Service példa:

Az `AuthService.cs` implementálja a regisztrációs és bejelentkezési logikát:

```
public class AuthService : IAuthService
{
    private readonly dbContext _context;
    private readonly IPasswordHasher _passwordHasher;
    private readonly ITokenGenerator _tokenGenerator;

    public async Task<AuthOperationResult> RegisterAsync(RegistrationRequest
userRequest)
    {
        var hashedPassword = _passwordHasher.HashPassword(userRequest.Password);
        var user = new User(userRequest, hashedPassword);

        _context.Users.Add(user);
        await _context.SaveChangesAsync();

        var userForResponse = new UserResponse(user);
        return new AuthOperationResult { Success = true, UserResponse =
userForResponse };
    }
}
```

Előnyök:

- **Újrafelhasználhatóság:** Az üzleti logika több controllerből is használható
- **Tesztelhetőség:** A service réteg külön tesztelhető
- **Separation of Concerns:** A controller csak a HTTP kérések kezelésére fókuszál

3.3.4. DTO (Data Transfer Object) Pattern

A DTO Pattern lehetővé teszi, hogy az adatokat strukturált formában küldjük a rétegek között, anélkül hogy a domain modelleket közvetlenül exponálnánk.

Request DTO-k:

A `RegistrationRequest.cs` egy DTO, amely a regisztrációs adatokat tartalmazza:

```
public record RegistrationRequest(string Email, string Nickname, string Password);
```

Response DTO-k:

A `UserResponse.cs` egy DTO, amely a felhasználói adatokat tartalmazza (jelszó hash nélkül):

```
public record UserResponse
{
    public int Id { get; init; }
    public string Email { get; init; }
    public string Nickname { get; init; }
    public string Description { get; init; }
    public string ImageLocation { get; init; }
}
```

Előnyök:

- **Biztonság:** Nem exponáljuk a belső domain modelleket (pl. jelszó hash)
- **Verziókezelés:** A DTO-k változtathatók anélkül, hogy a domain modelleket módosítanánk
- **Optimalizáció:** Csak a szükséges adatokat küldjük át a hálózaton

3.4. API végpontok áttekintése

Az `ApiPlan.md` fájl részletesen dokumentálja az összes API végpontot. Az alábbiakban összefoglaljuk a főbb végpontokat:

Autentikáció és felhasználókezelés:

- `POST /api/v1/Authentication/register` - Új felhasználó regisztrációja
- `POST /api/v1/Authentication/login` - Felhasználó bejelentkezés
- `GET /api/v1/User/{id}` - Felhasználó adatainak lekérdezése
- `PUT /api/v1/User/{id}` - Felhasználó adatainak frissítése
- `GET /api/v1/User/{id}/events` - Felhasználó eseményeinek lekérdezése

Autók kezelése:

- `POST /api/v1/Car/addCar/{userId}` - Új autó hozzáadása
- `PUT /api/v1/Car/updateCar/{carId}` - Autó frissítése
- `DELETE /api/v1/Car/deleteCar/{carId}` - Autó törlése
- `GET /api/v1/User/{id}/cars` - Felhasználó autóinak lekérdezése

Találkozók kezelése:

- `POST /api/v1/Meet/addMeet/{userId}` - Új találkozó létrehozása
- `PUT /api/v1/Meet/updateMeet/{meetId}` - Találkozó frissítése
- `DELETE /api/v1/Meet/deleteMeet/{meetId}` - Találkozó törlése
- `GET /api/v1/Meet/getMeet/{meetId}` - Találkozó részleteinek lekérdezése
- `GET /api/v1/Meet/getMeets` - Összes találkozó lekérdezése
- `GET /api/v1/Meet/getMeetsF` - Találkozók szűrt lekérdezése (helyszín, távolság, címkék)
- `PUT /api/v1/Meet/joinMeet/{meetId}/{userId}` - Csatlakozás találkozóhoz

Versenyek kezelése:

- `POST /api/v1/Race/addRace/{userId}` - Új verseny létrehozása
- `PUT /api/v1/Race/updateRace/{raceId}` - Verseny frissítése
- `DELETE /api/v1/Race/deleteRace/{raceId}` - Verseny törlése
- `GET /api/v1/Race/getRace/{raceId}` - Verseny részleteinek lekérdezése
- `GET /api/v1/Race/getRaces` - Összes verseny lekérdezése
- `GET /api/v1/Race/getRacesF` - Versenyek szűrt lekérdezése
- `PUT /api/v1/Race/joinRace/{raceId}/{userId}` - Csatlakozás versenyhez

Csapatok kezelése:

- `POST /api/v1/Crew/addCrew/{userId}` - Új crew létrehozása
- `PUT /api/v1/Crew/updateCrew/{crewId}` - Crew frissítése
- `GET /api/v1/Crew/getCrew/{crewId}` - Crew részleteinek lekérdezése
- `GET /api/v1/Crew/getCrews` - Összes crew lekérdezése
- `POST /api/v1/Crew/addUserToCrew` - Felhasználó hozzáadása crew-hoz

---

4. Adatbázis-tervezés

Az adatbázis tervezése kritikus fontosságú egy webes alkalmazás esetében, mivel az adatok struktúrája meghatározza a rendszer teljesítményét, skálázhatóságát és karbantarthatóságát. Ebben a fejezetben részletesen bemutatjuk a Rev_n_Roll platform adatbázis tervezését, az entitások azonosítását, a kapcsolatok definiálását és az Entity Framework Core konfigurációt.

4.1. Adatmodell tervezése

4.1.1. Entitások azonosítása

A követelményanalízis alapján öt fő entitást azonosítottunk, amelyek az alkalmazás domain modelljét alkotják:

1. User (Felhasználó):

A `User.cs` entitás reprezentálja a platformon regisztrált felhasználókat. Főbb tulajdonságai:

- `Id` (int): Elsődleges kulcs, auto-increment
- `Email` (string, max 320 karakter): Egyedi email cím, kötelező
- `Nickname` (string, max 32 karakter): Egyedi felhasználónév, kötelező
- `PasswordHash` (string, max 320 karakter): BCrypt hash-elt jelszó, kötelező
- `Description` (string): Felhasználói leírás, opcionális
- `ImageLocation` (string, max 64 karakter): Profilkép elérési útja, alapértelmezett: "default.jpg"

Navigációs tulajdonságok:

- `Cars` (List): Felhasználó autói (1:N kapcsolat)
- `UserCrews` (List): Felhasználó crew tagságai (N:M kapcsolat UserCrew-n keresztül)
- `Races` (List): Versenyek, amelyekre jelentkezett (N:M kapcsolat)
- `Meets` (List): Találkozók, amelyekre jelentkezett (N:M kapcsolat)

- `CreatedRaces` (List): Felhasználó által létrehozott versenyek (1:N kapcsolat)
- `CreatedMeets` (List): Felhasználó által létrehozott találkozók (1:N kapcsolat)

2. Car (Autó):

A `Car.cs` entitás reprezentálja a felhasználók autóit. Főbb tulajdonságai:

- `Id` (int): Elsődleges kulcs, auto-increment
- `UserId` (int): Idegen kulcs a User táblára, kötelező
- `Brand` (string, max 32 karakter): Autó márkája (pl. "Nissan"), kötelező
- `Model` (string, max 32 karakter): Autó modellje (pl. "GT-R"), kötelező
- `Engine` (string, max 32 karakter): Motor típusa (pl. "3.8L V6"), kötelező
- `HorsePower` (int): Teljesítmény lóerőben, kötelező
- `Description` (string): Autó leírása, opcionális

Navigációs tulajdonságok:

- `User` (User): A tulajdonos felhasználó (N:1 kapcsolat)

3. Crew (Csapat):

A `Crew.cs` entitás reprezentálja az autós csapatokat. Főbb tulajdonságai:

- `Id` (int): Elsődleges kulcs, auto-increment
- `Name` (string, max 32 karakter): Crew neve, egyedi, kötelező
- `Description` (string): Crew leírása, opcionális
- `ImageLocation` (string, max 64 karakter): Crew logó elérési útja, opcionális

Navigációs tulajdonságok:

- `UserCrews` (List): Crew tagjai (N:M kapcsolat UserCrew-n keresztül)
- `Meets` (List): Crew által szervezett találkozók (1:N kapcsolat)

4. Meet (Találkozó):

A `Meet.cs` entitás reprezentálja az autós találkozókat. Főbb tulajdonságai:

- `Id` (int): Elsődleges kulcs, auto-increment
- `Name` (string, max 64 karakter): Találkozó neve, kötelező
- `Description` (string): Találkozó leírása, opcionális
- `CreatorId` (int): Idegen kulcs a User táblára (létrehozó), kötelező
- `CrewId` (int?): Idegen kulcs a Crew táblára, nullable (opcionális crew szervezés)
- `Location` (string, max 128 karakter): Helyszín szöveges leírása, kötelező
- `Coordinates` (string, max 64 karakter): GPS koordináták "latitude,longitude" formátumban, kötelező
- `Date` (DateTime): Találkozó dátuma és időpontja, kötelező
- `Private` (bool): Privát vagy nyilvános esemény, kötelező
- `Tags` (List): Találkozó címkéi (enum lista), opcionális

Navigációs tulajdonságok:

- `Creator` (User): Létrehozó felhasználó (N:1 kapcsolat)
- `Crew` (Crew?): Szervező crew, nullable (N:1 kapcsolat)

- `Users` (List): Jelentkezett felhasználók (N:M kapcsolat)

Számított tulajdonságok (NotMapped):

- `Latitude` (double): Szélesség a koordinátákból kinyerve
- `Longitude` (double): Hosszúság a koordinátákból kinyerve

5. Race (Verseny):

A `Race.cs` entitás reprezentálja az autós versenyeket. Főbb tulajdonságai:

- `Id` (int): Elsődleges kulcs, auto-increment
- `Name` (string, max 64 karakter): Verseny neve, kötelező
- `Description` (string): Verseny leírása, opcionális
- `CreatorId` (int): Idegen kulcs a User táblára (létrehozó), kötelező
- `CrewId` (int?): Idegen kulcs a Crew táblára, nullable
- `RaceType` (RaceType enum): Verseny típusa (Drag, Circuit, Drift, Rally), kötelező
- `Location` (string, max 128 karakter): Helyszín szöveges leírása, kötelező
- `Coordinates` (string, max 32 karakter): GPS koordináták, kötelező
- `Private` (bool): Privát vagy nyilvános esemény, kötelező
- `Date` (DateTime): Verseny dátuma és időpontja, kötelező

Navigációs tulajdonságok:

- `Creator` (User): Létrehozó felhasználó (N:1 kapcsolat)
- `Crew` (Crew?): Szervező crew, nullable (N:1 kapcsolat)
- `Users` (List): Jelentkezett felhasználók (N:M kapcsolat)

Számított tulajdonságok (NotMapped):

- `Latitude` (double): Szélesség a koordinátákból kinyerve
- `Longitude` (double): Hosszúság a koordinátákból kinyerve

6. UserCrew (Kapcsolótábla):

A `UserCrew.cs` entitás egy explicit kapcsolótábla a User és Crew entitások között, amely tartalmazza a felhasználó rangját a crew-ban:

- `Id` (int): Elsődleges kulcs, auto-increment
- `UserId` (int): Idegen kulcs a User táblára, kötelező
- `CrewId` (int): Idegen kulcs a Crew táblára, kötelező
- `Rank` (Rank enum): Felhasználó rangja (Leader, CoLeader, Recruiter, Member), kötelező

Navigációs tulajdonságok:

- `User` (User): A felhasználó (N:1 kapcsolat)
- `Crew` (Crew): A crew (N:1 kapcsolat)

4.1.2. Kapcsolatok definiálása

Az entitások között komplex kapcsolatok állnak fenn, amelyeket az Entity Framework Core Fluent API-val konfigurálunk a `dbContext.cs` fájlban.

1:N kapcsolatok:

- **User → Car:** Egy felhasználónak több autója lehet, de egy autó csak egy felhasználóhoz tartozik.

```
// Implicit konfiguráció a Car.UserId foreign key miatt
```

- **User → CreatedMeets:** Egy felhasználó több találkozót hozhat létre, de egy találkozónak csak egy létrehozója van.

```
modelBuilder.Entity<Meet>()
    .HasOne(m => m.Creator)
    .WithMany(u => u.CreatedMeets)
    .HasForeignKey(m => m.CreatorId);
```

- **User → CreatedRaces:** Egy felhasználó több versenyt hozhat létre, de egy versenynek csak egy létrehozója van.

```
modelBuilder.Entity<Race>()
    .HasOne(r => r.Creator)
    .WithMany(u => u.CreatedRaces)
    .HasForeignKey(r => r.CreatorId);
```

- **Crew → Meets:** Egy crew több találkozót szervezhet, de egy találkozót csak egy crew szervezhet (vagy egyik sem).

```
modelBuilder.Entity<Meet>()
    .HasOne(m => m.Crew)
    .WithMany(c => c.Meets)
    .HasForeignKey(m => m.CrewId)
    .OnDelete(DeleteBehavior.SetNull);
```

  A `OnDelete(DeleteBehavior.SetNull)` beállítás biztosítja, hogy ha egy crew törlődik, a találkozók megmaradnak, de a `CrewId` null-ra állítódik.

N:M kapcsolatok (implicit kapcsolótáblával):

- **User ↔ Meet (résztvevők):** Egy felhasználó több találkozóra jelentkezhet, és egy találkozón több felhasználó részt vehet.

```
modelBuilder.Entity<User>()
    .HasMany(u => u.Meets)
    .WithMany(m => m.Users)
    .UsingEntity(j => j.ToTable("UserMeet"));
```

Az EF Core automatikusan létrehoz egy `UserMeet` kapcsolótáblát `UserId` és `MeetId` oszlopokkal.

- **User ↔ Race (résztvevők):** Egy felhasználó több versenyre jelentkezhet, és egy versenyen több felhasználó részt vehet.

```
modelBuilder.Entity<User>()
    .HasMany(u => u.Races)
    .WithMany(r => r.Users)
    .UsingEntity(j => j.ToTable("UserRace"));
```

N:M kapcsolat (explicit kapcsolótáblával):

- **User ↔ Crew (tagság rangokkal):** Egy felhasználó több crew tagja lehet, és egy crew-nak több tagja van. A kapcsolótábla (`UserCrew`) tartalmazza a felhasználó rangját.

```
modelBuilder.Entity<UserCrew>()
    .HasOne(uc => uc.User)
    .WithMany(u => u.UserCrews)
    .HasForeignKey(uc => uc.UserId);

modelBuilder.Entity<UserCrew>()
    .HasOne(uc => uc.Crew)
    .WithMany(c => c.UserCrews)
    .HasForeignKey(uc => uc.CrewId);
```

4.1.3. Egyedi megszorítások (Unique Constraints)

Az adatintegritás biztosítása érdekében egyedi megszorításokat definiálunk bizonyos oszlopokra:

```
// User email egyedi kell legyen
modelBuilder.Entity<User>()
    .HasIndex(u => u.Email)
    .IsUnique();

// User nickname egyedi kell legyen
modelBuilder.Entity<User>()
    .HasIndex(u => u.Nickname)
    .IsUnique();

// Crew name egyedi kell legyen
modelBuilder.Entity<Crew>()
    .HasIndex(c => c.Name)
    .IsUnique();
```

Ezek az indexek nemcsak az egyediséget biztosítják, hanem javítják a keresési teljesítményt is.

4.2. Entity Framework Core konfiguráció

4.2.1. DbContext implementáció

A `dbContext.cs` osztály az Entity Framework Core központi osztálya, amely az adatbázis kapcsolatot és a DbSet-eket kezeli:

```
public class dbContext : DbContext
{
    public DbSet<Car> Cars { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Crew> Crews { get; set; }
    public DbSet<UserCrew> UserCrews { get; set; }
    public DbSet<Race> Races { get; set; }
    public DbSet<Meet> Meets { get; set; }

    public dbContext(DbContextOptions<dbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Fluent API konfigurációk...
    }
}
```

DbSet-ek:

Minden DbSet egy táblát reprezentál az adatbázisban. Az EF Core automatikusan létrehozza a táblákat a DbSet-ek alapján, a tábla neve megegyezik a DbSet nevével (pl. `Cars`, `Users`, `Crews`).

Constructor Injection:

A `dbContext` konstruktora `DbContextOptions<dbContext>` paramétert vár, amelyet a Dependency Injection konténer injektál. Ez lehetővé teszi a connection string és egyéb beállítások konfigurálását a `Program.cs` fájlban:

```
builder.Services.AddDbContext<dbContext>(options =>
    options.UseNpgsql(connectionString));
```

4.2.2. Fluent API használata

A Fluent API egy alternatív módja az entitások konfigurálásának a Data Annotations mellett. A Fluent API előnyei:

- **Centralizált konfiguráció:** Minden konfiguráció egy helyen van (OnModelCreating metódus)
- **Több lehetőség:** Bizonyos konfigurációk csak Fluent API-val érhetők el

- **Tiszta domain modellek:** A domain osztályok nem tartalmaznak infrastruktúra-specifikus annotációkat

Példa - Kapcsolatok konfigurálása:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // User-Meet many-to-many kapcsolat
    modelBuilder.Entity<User>()
        .HasMany(u => u.Meets)
        .WithMany(m => m.Users)
        .UsingEntity(j => j.ToTable("UserMeet"));

    // Meet-Creator one-to-many kapcsolat
    modelBuilder.Entity<Meet>()
        .HasOne(m => m.Creator)
        .WithMany(u => u.CreatedMeets)
        .HasForeignKey(m => m.CreatorId);

    // Meet-Crew one-to-many kapcsolat nullable foreign key-vel
    modelBuilder.Entity<Meet>()
        .HasOne(m => m.Crew)
        .WithMany(c => c.Meets)
        .HasForeignKey(m => m.CrewId)
        .OnDelete(DeleteBehavior.SetNull);
}
```

Példa - Egyedi indexek konfigurálása:

```
modelBuilder.Entity<User>()
    .HasIndex(u => u.Email)
    .IsUnique();
```

4.2.3. Migrációk kezelése

Az Entity Framework Core migrációs rendszere lehetővé teszi az adatbázis séma verziókezelését. A migrációk C# kódként tárolódnak, így verziókezelhetők és megoszthatók a csapattagokkal.

Migráció létrehozása:

Amikor módosítjuk az entitásokat vagy a Fluent API konfigurációt, új migrációt hozunk létre:

```
dotnet ef migrations add AddMeetTagsColumn --project ThesisBackend.Data
```

Ez a parancs létrehoz egy új migrációs fájlt a `Migrations` mappában, amely tartalmazza az `Up` és `Down` metódusokat:

```
public partial class AddMeetTagsColumn : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<List<int>>(
            name: "Tags",
            table: "Meets",
            nullable: true);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "Tags",
            table: "Meets");
    }
}
```

Migráció alkalmazása:

A migrációt az adatbázisra alkalmazzuk:

```
dotnet ef database update --project ThesisBackend.Data
```

Ez a parancs végrehajtja az összes függőben lévő migrációt az adatbázison.

Migrációk előnyei:

- **Verziókezelés:** Az adatbázis séma változásai követhetők
- **Csapatmunka:** A migrációk megoszthatók a csapattagokkal
- **Rollback:** A `Down` metódus lehetővé teszi a változások visszavonását
- **Automatizálás:** A migrációk automatikusan alkalmazhatók deployment során

4.3. Adatbázis séma

4.3.1. Táblák struktúrája

Az Entity Framework Core a domain modellek alapján automatikusan generálja az adatbázis sémát. Az alábbiakban bemutatjuk a főbb táblák struktúráját:

Users tábla:

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
Email	VARCHAR(320)	NOT NULL, UNIQUE
Nickname	VARCHAR(32)	NOT NULL, UNIQUE

Oszlop	Típus	Megszorítások
PasswordHash	VARCHAR(320)	NOT NULL
Description	TEXT	NULL
ImageLocation	VARCHAR(64)	NULL, DEFAULT 'default.jpg'

Cars tábla:

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
UserId	INTEGER	NOT NULL, FOREIGN KEY → Users(Id)
Brand	VARCHAR(32)	NOT NULL
Model	VARCHAR(32)	NOT NULL
Engine	VARCHAR(32)	NOT NULL
HorsePower	INTEGER	NOT NULL
Description	TEXT	NULL

Crews tábla:

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(32)	NOT NULL, UNIQUE
Description	TEXT	NULL
ImageLocation	VARCHAR(64)	NULL

Meets tábla:

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(64)	NOT NULL
Description	TEXT	NULL
CreatorId	INTEGER	NOT NULL, FOREIGN KEY → Users(Id)
CrewId	INTEGER	NULL, FOREIGN KEY → Crews(Id) ON DELETE SET NULL
Location	VARCHAR(128)	NOT NULL
Coordinates	VARCHAR(64)	NOT NULL
Date	TIMESTAMP	NOT NULL

Oszlop	Típus	Megszorítások
Private	BOOLEAN	NOT NULL
Tags	INTEGER[]	NULL (PostgreSQL array)

Races tábla:

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(64)	NOT NULL
Description	TEXT	NULL
CreatorId	INTEGER	NOT NULL, FOREIGN KEY → Users(Id)
CrewId	INTEGER	NULL, FOREIGN KEY → Crews(Id) ON DELETE SET NULL
RaceType	INTEGER	NOT NULL (enum: 0=Drag, 1=Circuit, 2=Drift, 3=Rally)
Location	VARCHAR(128)	NOT NULL
Coordinates	VARCHAR(32)	NOT NULL
Date	TIMESTAMP	NOT NULL
Private	BOOLEAN	NOT NULL

UserCrews tábla (explicit kapcsolótábla):

Oszlop	Típus	Megszorítások
Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
UserId	INTEGER	NOT NULL, FOREIGN KEY → Users(Id)
CrewId	INTEGER	NOT NULL, FOREIGN KEY → Crews(Id)
Rank	INTEGER	NOT NULL (enum: 0=Leader, 1=CoLeader, 2=Recruiter, 3=Member)

UserMeet tábla (implicit kapcsolótábla):

Oszlop	Típus	Megszorítások
UsersId	INTEGER	PRIMARY KEY, FOREIGN KEY → Users(Id)
MeetsId	INTEGER	PRIMARY KEY, FOREIGN KEY → Meets(Id)

UserRace tábla (implicit kapcsolótábla):

Oszlop	Típus	Megszorítások
UsersId	INTEGER	PRIMARY KEY, FOREIGN KEY → Users(Id)
RacesId	INTEGER	PRIMARY KEY, FOREIGN KEY → Races(Id)

4.3.2. Indexek és teljesítmény optimalizáció

Az adatbázis teljesítményének javítása érdekében indexeket hozunk létre a gyakran keresett oszlopokon:

Egyedi indexek (Unique Indexes):

```
CREATE UNIQUE INDEX IX_Users_Email ON Users(Email);
CREATE UNIQUE INDEX IX_Users_Nickname ON Users(Nickname);
CREATE UNIQUE INDEX IX_Crews_Name ON Crews(Name);
```

Ezek az indexek biztosítják az egyediséget és gyorsítják a keresést email, nickname és crew név alapján.

Foreign Key indexek:

Az EF Core automatikusan létrehoz indexeket a foreign key oszlopokon, ami javítja a JOIN műveletek teljesítményét:

```
CREATE INDEX IX_Cars_UserId ON Cars(UserId);
CREATE INDEX IX_Meets_CreatorId ON Meets(CreatorId);
CREATE INDEX IX_Meets_CrewId ON Meets(CrewId);
CREATE INDEX IX_Races_CreatorId ON Races(CreatorId);
CREATE INDEX IX_Races_CrewId ON Races(CrewId);
```

Kompozit indexek (jövőbeli optimalizáció):

Amennyiben gyakran keresünk találkozókat dátum és helyszín alapján, kompozit indexet hozhatunk létre:

```
CREATE INDEX IX_Meets_Date_Location ON Meets(Date, Location);
```

4.3.3. Adatintegritás és megszorítások

Az adatintegritás biztosítása érdekében több szintű megszorításokat alkalmazunk:

1. Adatbázis szintű megszorítások:

- **NOT NULL:** Kötelező mezők (pl. Email, Nickname, PasswordHash)
- **UNIQUE:** Egyedi mezők (pl. Email, Nickname, Crew Name)
- **FOREIGN KEY:** Referenciális integritás (pl. Car.UserId → User.Id)
- **CHECK:** Értéktartomány ellenőrzés (pl. HorsePower > 0)

2. Alkalmazás szintű validáció:

A FluentValidation könyvtár segítségével validáljuk az adatokat, mielőtt az adatbázisba kerülnének. Például a `RegistrationRequestValidator.cs` ellenőrzi:

- Email formátum helyessége
- Nickname hossza (max 32 karakter)

- Jelszó erőssége (min 8 karakter, speciális karakter, szám)
- Email és nickname egyedisége (adatbázis lekérdezéssel)

3. Domain szintű validáció:

A domain modellek Data Annotations attribútumokkal rendelkeznek:

```
[Required]
[StringLength(320)]
public string Email { get; set; }
```

Ez biztosítja, hogy az EF Core megfelelő adatbázis megszorításokat generáljon.

4.4. Adatbázis kapcsolat kezelése

4.4.1. Connection String konfiguráció

A connection string az `appsettings.json` fájlban van definiálva:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Host=localhost;Database=RevNRollDb;Username=postgres;Password=yourpassword"
  }
}
```

A `Program.cs` fájlban olvassuk be a connection string-et:

```
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<dbContext>(options =>
    options.UseNpgsql(connectionString));
```

Környezeti változók használata:

Production környezetben a connection string környezeti változóból olvasódik be, így nem kerül a kódba:

```
export ConnectionStrings__DefaultConnection="Host=prod-db;Database=RevNRollDb;..."
```

4.4.2. Connection Pooling

Az Entity Framework Core automatikusan connection pooling-ot használ, ami újrafelhasználja az adatbázis kapcsolatokat, csökkentve ezzel a kapcsolat létrehozás overhead-jét.

Előnyök:

- **Teljesítmény:** Gyorsabb adatbázis műveletek
- **Skálázhatóság:** Több egyidejű kérés kiszolgálása
- **Erőforrás-hatékonyság:** Kevesebb adatbázis kapcsolat

Konfiguráció:

A connection pooling paraméterei a connection string-ben állíthatók:

```
Host=localhost;Database=RevNRollDb;Username=postgres;Password=yourpassword;Pooling
=true;MinPoolSize=5;MaxPoolSize=100
```

4.4.3. Tranzakció kezelés

Az Entity Framework Core automatikusan tranzakcióba csomagolja a `SaveChangesAsync()` hívásokat. Ha több műveletet szeretnénk egy tranzakcióban végrehajtani, explicit tranzakciót használhatunk:

```
using var transaction = await _context.Database.BeginTransactionAsync();
try
{
    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    _context.Cars.Add(car);
    await _context.SaveChangesAsync();

    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Ez biztosítja az ACID tulajdonságokat (Atomicity, Consistency, Isolation, Durability).

---

5. Backend Implementáció

A Rev_n_Roll platform backend implementációja .NET 9 és ASP.NET Core technológiákra épül, Clean Architecture elveket követve. Ebben a fejezetben részletesen bemutatjuk a backend főbb komponenseit, az autentikációs és autorizációs mechanizmusokat, a validációt, az API végpontokat, valamint a hibakezelést és naplózást.

5.1. Projekt struktúra

A backend projekt négy fő rétegre oszlik, amelyek a Clean Architecture elveit követik:

1. ThesisBackend.Domain (Core réteg):

Ez a legbelső réteg, amely nem függ semmilyen külső library-től vagy keretrendszertől. Tartalmazza:

- **Models mappa:** Domain entitások (`User.cs`, `Car.cs`, `Crew.cs`, `Meet.cs`, `Race.cs`, `UserCrew.cs`)
- **Messages mappa:** Request és Response DTO-k (pl. `RegistrationRequest.cs`, `UserResponse.cs`)
- **Enums:** Felsorolási típusok (pl. `MeetTags`, `RaceType`, `Rank`)

2. ThesisBackend.Data (Infrastructure réteg):

Ez a réteg felelős az adatelérésért. Tartalmazza:

- `dbContext.cs`: Entity Framework Core DbContext
- **Migrations mappa:** Adatbázis migrációk
- `ConnectionString.cs`: Connection string konfiguráció

3. ThesisBackend.Services (Application réteg):

Ez a réteg tartalmazza az üzleti logikát. Tartalmazza:

- **Authentication mappa:**
  - **Services:** `AuthService.cs`, `PasswordHasher.cs`, `TokenGenerator.cs`
  - **Interfaces:** `IAuthService`, `IPasswordHasher`, `ITokenGenerator`
  - **Validators:** `RegistrationRequestValidator.cs`, `LoginRequestValidator.cs`
  - **Models:** `JwtSettings.cs`, `AuthOperationResult`
- **UserService, CarService, MeetService, RaceService, CrewService mappák:** Hasonló struktúrával (Services, Interfaces, Validators)

4. ThesisBackend (Presentation réteg):

Ez a réteg felelős a HTTP kérések fogadásáért és válaszok küldéséért. Tartalmazza:

- **Controllers mappa:** API controllerek (`AuthController.cs`, `UserController.cs`, `CarController.cs`, `MeetController.cs`, `RaceController.cs`, `CrewController.cs`)
- `Program.cs`: Alkalmazás belépési pont és konfiguráció
- `appsettings.json`: Alkalmazás beállítások

5.2. Autentikáció és autorizáció

5.2.1. JWT token alapú autentikáció

A Rev_n_Roll platform JWT (JSON Web Token) alapú autentikációt használ. A JWT egy kompakt, URL-safe token formátum, amely lehetővé teszi a felhasználók biztonságos azonosítását.

JWT struktúra:

Egy JWT három részből áll, amelyeket pont (`.`) karakter választ el:

1. **Header:** Tartalmazza a token típusát (JWT) és a használt hash algoritmust (HS256)
2. **Payload:** Tartalmazza a claims-eket (felhasználói adatok, lejárati idő)
3. **Signature:** A header és payload digitális aláírása a secret key-vel

TokenGenerator implementáció:

A `TokenGenerator.cs` osztály felelős a JWT tokenek generálásáért:

```
public class TokenGenerator : ITokenGenerator
{
    private readonly JwtSettings _jwtSettings;
    private readonly ILogger<TokenGenerator> _logger;

    public string GenerateAccessToken(User user)
    {
        _logger.LogInformation("Generating JWT for UserID: {userid}, Email:
{email}",
            user.Id, user.Email);

        var tokenHandler = new JwtSecurityTokenHandler();
        var secret = Encoding.ASCII.GetBytes(_jwtSettings.Secret);

        var tokenDescriptor = new SecurityTokenDescriptor()
        {
            Subject = new ClaimsIdentity(new Claim[]
            {
                new Claim(ClaimTypes.Name, user.Id.ToString())
            }),
            Expires = DateTime.UtcNow.AddMinutes(_jwtSettings.DurationInMinutes),
            Issuer = _jwtSettings.Issuer,
            Audience = _jwtSettings.Audience,
            SigningCredentials = new SigningCredentials(
                new SymmetricSecurityKey(secret),
                SecurityAlgorithms.HmacSha256Signature)
        };

        var token = tokenHandler.CreateToken(tokenDescriptor);
        return tokenHandler.WriteToken(token);
    }
}
```

JWT konfiguráció:

A JWT beállítások az `appsettings.json` fájlban vannak definiálva:

```
{
  "Jwt": {
    "Secret": "YourSuperSecretKeyThatIsAtLeast32CharactersLong!",
    "Issuer": "RevNRollBackend",
    "Audience": "RevNRollFrontend",
    "DurationInMinutes": 60
  }
}
```

JWT validáció:

A `Program.cs` fájlban konfiguráljuk a JWT autentikációt:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        var jwtSettings = builder.Configuration.GetSection("Jwt").Get<JwtSettings>
();

        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtSettings.Secret))
        };

        options.Events = new JwtBearerEvents
        {
            OnMessageReceived = context =>
            {
                var token = context.Request.Cookies["accessToken"];
                if (!string.IsNullOrEmpty(token))
                {
                    context.Token = token;
                }
                return Task.CompletedTask;
            }
        };
    });
```

A `OnMessageReceived` event handler lehetővé teszi, hogy a JWT tokent HttpOnly cookie-ból olvassuk ki, ami biztonságosabb, mint a localStorage használata (XSS támadások ellen véd).

5.2.2. Jelszó hashelés BCrypt-tel

A felhasználói jelszavakat soha nem tároljuk plain text formában. A `PasswordHasher.cs` osztály BCrypt algoritmust használ a jelszavak hash-eléséhez:

```
public class PasswordHasher : IPasswordHasher
{
    private readonly ILogger<PasswordHasher> _logger;

    public string HashPassword(string password)
    {
```

```
        _logger.LogDebug("Hashing password.");
        if (string.IsNullOrEmpty(password))
        {
            _logger.LogDebug("Failed to hash password: Password is null or
empty.");
            return string.Empty;
        }
        var hashedPassword = BCrypt.Net.BCrypt.HashPassword(password);
        _logger.LogDebug("Password hashed successfully.");
        return hashedPassword;
    }

    public bool VerifyPassword(string password, string hashedPassword)
    {
        _logger.LogDebug("Verifying password.");
        if (string.IsNullOrEmpty(password) ||
string.IsNullOrEmpty(hashedPassword))
        {
            _logger.LogDebug("Failed to verify password: Password or hashed
password is null or empty.");
            return false;
        }
        var isValid = BCrypt.Net.BCrypt.Verify(password, hashedPassword);
        _logger.LogDebug("Password verification result: {VerificationResult}",
isValid);
        return isValid;
    }
}
```

BCrypt előnyei:

- **Adaptive hashing:** A work factor növelhető, így a jövőben is biztonságos marad
- **Salt automatikus generálása:** Minden jelszóhoz egyedi salt generálódik
- **Lassú hash:** Szándékosan lassú, ami megnehezíti a brute-force támadásokat
- **Iparági szabvány:** Széles körben használt és tesztelt algoritmus

5.2.3. HttpOnly cookie használata

A JWT tokent HttpOnly cookie-ban tároljuk, ami több biztonsági előnnyel jár:

Előnyök:

- **XSS védelem:** JavaScript nem férhet hozzá a cookie-hoz, így XSS támadás esetén sem lopható el a token
- **Automatikus küldés:** A böngésző automatikusan elküldi a cookie-t minden kéréssel
- **CSRF védelem:** SameSite=Lax beállítással védhetünk CSRF támadások ellen

Cookie beállítások:

Az `AuthController.cs` fájlban a login végpont beállítja a cookie-t:

```
Response.Cookies.Append("accessToken", result.Token, new CookieOptions
{
    HttpOnly = true,           // JavaScript nem férhet hozzá
    Secure = true,             // Csak HTTPS-en keresztül küldhető
    SameSite = SameSiteMode.Lax, // CSRF védelem
    Expires = DateTimeOffset.UtcNow.AddMinutes(_jwtSettings.DurationInMinutes),
    Path = "/"                 // Cookie elérhető az összes útvonalhoz
});
```

5.3. Validáció FluentValidation-nel

A FluentValidation egy népszerű .NET library, amely lehetővé teszi a validációs szabályok tiszta és olvasható módon történő definiálását.

5.3.1. RegistrationRequestValidator

A `RegistrationRequestValidator.cs` validálja a regisztrációs kéréseket:

```
public class RegistrationRequestValidator : AbstractValidator<RegistrationRequest>
{
    private readonly dbContext _context;

    public RegistrationRequestValidator(dbContext context)
    {
        _context = context;

        RuleFor(user => user.Nickname)
            .NotEmpty().WithMessage("Nickname is required.")
            .Length(3, 32).WithMessage("Nickname must be between 3 and 32
characters.")
            .MustAsync(BeUniqueNicknameAsync).WithMessage("Nickname already
taken.");

        RuleFor(user => user.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("A valid email address is required.")
            .MaximumLength(320).WithMessage("Email must not exceed 320
characters.")
            .MustAsync(BeUniqueEmailAsync).WithMessage("An account with this email
already exists.");

        RuleFor(user => user.Password)
            .NotEmpty().WithMessage("Password is required.")
            .MinimumLength(8).WithMessage("Password must be at least 8 characters
long.")
            .Matches(@"[A-Z]").WithMessage("Password must contain at least one
uppercase letter.")
            .Matches(@"[a-z]").WithMessage("Password must contain at least one
lowercase letter.")
            .Matches(@"[0-9]").WithMessage("Password must contain at least one
```

```
digit.")
            .Matches(@"[^a-zA-Z0-9]").WithMessage("Password must contain at least
one special character.");
    }

    private async Task<bool> BeUniqueNicknameAsync(string nickname,
CancellationToken cancellationToken)
    {
        return !await _context.Users.AnyAsync(u => u.Nickname == nickname,
cancellationToken);
    }

    private async Task<bool> BeUniqueEmailAsync(string email, CancellationToken
cancellationToken)
    {
        return !await _context.Users.AnyAsync(u => u.Email == email,
cancellationToken);
    }
}
```

Validációs szabályok:

- **Nickname:** 3-32 karakter, egyedi
- **Email:** Érvényes email formátum, max 320 karakter, egyedi
- **Password:** Min 8 karakter, tartalmaz nagybetűt, kisbetűt, számot és speciális karaktert

Aszinkron validáció:

A `MustAsync` metódus lehetővé teszi aszinkron validációs szabályok definiálását, például adatbázis lekérdezéseket az egyediség ellenőrzésére.

5.3.2. Validáció használata a controllerekben

Az `AuthController.cs` fájlban a validátort Dependency Injection-nel injektáljuk:

```
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;
    private readonly IValidator<RegistrationRequest>
_registrationRequestValidator;
    private readonly IValidator<LoginRequest> _loginRequestValidator;

    [HttpPost("register")]
    [AllowAnonymous]
    public async Task<IActionResult> Register(RegistrationRequest
registrationRequest)
    {
        _logger.LogInformation("Received registration request for user: {nickname}
with email: {email}",
            registrationRequest.Nickname, registrationRequest.Email);
```

```
        var validationResult = await
_registrationRequestValidator.ValidateAsync(registrationRequest);

        if (!validationResult.IsValid)
        {
            validationResult.AddToModelState(ModelState);
            _logger.LogWarning("Registration request validation failed for user:
{nickname} with email: {email}, with errors: {errors}",
                registrationRequest.Nickname, registrationRequest.Email,
validationResult.ToDictionary());
            return ValidationProblem(ModelState);
        }

        var result = await _authService.RegisterAsync(registrationRequest);
        if (!result.Success)
        {
            _logger.LogError("Registration failed for user: {nickname} with email:
{email}, error: {error}",
                registrationRequest.Nickname, registrationRequest.Email,
result.ErrorMessage);
            return Problem(
                statusCode: 400,
                title: "Registration Failed",
                detail: result.ErrorMessage ?? "An error occurred during
registration."
            );
        }

        _logger.LogInformation("User registered successfully: {nickname} with
email: {email}",
            registrationRequest.Nickname, registrationRequest.Email);
        return Ok(new { message = "User registered successfully" });
    }
}
```

Validációs folyamat:

1. A controller fogadja a HTTP kérést
2. A validátor ellenőrzi a kérés adatait
3. Ha a validáció sikertelen, `ValidationProblem` választ küldünk (400 Bad Request)
4. Ha a validáció sikeres, továbbítjuk a kérést a service rétegnek

5.4. Service réteg implementáció

5.4.1. AuthService

Az `AuthService.cs` implementálja a regisztrációs és bejelentkezési logikát:

```
public class AuthService : IAuthService
{
    private readonly dbContext _context;
```

```
    private readonly IPasswordHasher _passwordHasher;
    private readonly ITokenGenerator _tokenGenerator;
    private readonly ILogger<AuthService> _logger;

    public async Task<AuthOperationResult> RegisterAsync(RegistrationRequest
userRequest)
    {
        _logger.LogInformation("Attempting to register user with Email: {email},
Nickname: {nickname}",
            userRequest.Email, userRequest.Nickname);

        _logger.LogDebug("Hashing password for user {nickname}.",
userRequest.Nickname);
        var hashedPassword = _passwordHasher.HashPassword(userRequest.Password);
        var user = new User(userRequest, hashedPassword);

        _context.Users.Add(user);
        await _context.SaveChangesAsync();

        _logger.LogInformation("User {nickname} (ID: {userid}) registered
successfully.",
            user.Nickname, user.Id);

        var userForResponse = new UserResponse(user);
        return new AuthOperationResult { Success = true, UserResponse =
userForResponse };
    }

    public async Task<AuthOperationResult> LoginAsync(LoginRequest request)
    {
        _logger.LogInformation("Attempting to login with email {email}.",
request.Email);
        var user = await _context.Users.FirstOrDefaultAsync(u => u.Email ==
request.Email);

        if (user == null || !_passwordHasher.VerifyPassword(request.Password,
user.PasswordHash))
        {
            _logger.LogWarning("Login failed for email {email}. User not found or
password mismatch.",
                request.Email);
            return new AuthOperationResult {
                Success = false,
                ErrorMessage = "Invalid email or password."
            };
        }

        _logger.LogDebug("Password verification successful for UserID: {userid}.",
user.Id);
        _logger.LogInformation("Initiating token generation for UserID:
{userid}.", user.Id);

        var token = _tokenGenerator.GenerateAccessToken(user);
        var userForResponse = new UserResponse(user);
```

```
        return new AuthOperationResult {
            Success = true,
            UserResponse = userForResponse,
            Token = token
        };
    }
}
```

Service réteg felelősségei:

- **Üzleti logika végrehajtása:** Jelszó hashelés, token generálás
- **Adatbázis műveletek koordinálása:** User létrehozása, mentése
- **Hibakezelés:** Sikertelen login esetén hibaüzenet visszaadása
- **Naplózás:** Minden fontos esemény naplózása

5.4.2. MeetService példa

A találkozók kezelését a `MeetService` osztály végzi. Példa a találkozó létrehozására:

```
public async Task<MeetOperationResult> AddMeetAsync(MeetRequest meetRequest, int
userId)
{
    _logger.LogInformation("Adding meet for user {UserId}", userId);

    var user = await _context.Users.FindAsync(userId);
    if (user == null)
    {
        return new MeetOperationResult {
            Success = false,
            ErrorMessage = "User not found"
        };
    }

    Crew? crew = null;
    if (meetRequest.CrewId.HasValue)
    {
        crew = await _context.Crews.FindAsync(meetRequest.CrewId.Value);
    }

    var meet = new Meet(meetRequest, user, crew);
    _context.Meets.Add(meet);
    await _context.SaveChangesAsync();

    var meetResponse = new MeetResponse(meet);
    return new MeetOperationResult {
        Success = true,
        MeetResponse = meetResponse
    };
}
```

5.5. API végpontok implementáció

5.5.1. AuthController

Az `AuthController.cs` kezeli az autentikációs végpontokat:

Register végpont:

```
[HttpPost("register")]
[AllowAnonymous]
public async Task<IActionResult> Register(RegistrationRequest registrationRequest)
{
    // Validáció
    var validationResult = await
_registrationRequestValidator.ValidateAsync(registrationRequest);
    if (!validationResult.IsValid)
    {
        validationResult.AddToModelState(ModelState);
        return ValidationProblem(ModelState);
    }

    // Service hívás
    var result = await _authService.RegisterAsync(registrationRequest);
    if (!result.Success)
    {
        return Problem(
            statusCode: 400,
            title: "Registration Failed",
            detail: result.ErrorMessage ?? "An error occurred during
registration."
        );
    }

    return Ok(new { message = "User registered successfully" });
}
```

Login végpont:

```
[HttpPost("login")]
[AllowAnonymous]
public async Task<IActionResult> Login(LoginRequest loginRequest)
{
    // Validáció
    var validationResult = await
_loginRequestValidator.ValidateAsync(loginRequest);
    if (!validationResult.IsValid)
    {
        validationResult.AddToModelState(ModelState);
        return ValidationProblem(ModelState);
    }
```

```
    // Service hívás
    var result = await _authService.LoginAsync(loginRequest);
    if (!result.Success || string.IsNullOrEmpty(result.Token) ||
result.UserResponse == null)
    {
        return Problem(
            statusCode: 401,
            title: "Login Failed",
            detail: result.ErrorMessage ?? "Invalid username or password."
        );
    }

    // JWT token beállítása HttpOnly cookie-ban
    Response.Cookies.Append("accessToken", result.Token, new CookieOptions
    {
        HttpOnly = true,
        Secure = true,
        SameSite = SameSiteMode.Lax,
        Expires =
DateTimeOffset.UtcNow.AddMinutes(_jwtSettings.DurationInMinutes),
        Path = "/"
    });

    return Ok(result.UserResponse);
}
```

5.5.2. MeetController

A `MeetController.cs` kezeli a találkozókkal kapcsolatos végpontokat:

```
[Route("api/v1/Meet")]
[ApiController]
public class MeetController : ControllerBase
{
    private readonly IMeetService _meetService;
    private readonly IValidator<MeetRequest> _meetRequestValidator;
    private readonly ILogger<MeetController> _logger;

    [HttpPost("addMeet/{userId}")]
    public async Task<ActionResult<MeetResponse>> AddMeet(MeetRequest meetRequest,
int userId)
    {
        _logger.LogInformation("AddMeet request for user {UserId}", userId);

        var validationResult = await
_meetRequestValidator.ValidateAsync(meetRequest);
        if (!validationResult.IsValid)
        {
            return BadRequest(validationResult.Errors);
        }
```

```
        var result = await _meetService.AddMeetAsync(meetRequest, userId);
        return result.Success ? Ok(result.MeetResponse) :
Problem(result.ErrorMessage);
    }

    [HttpGet("getMeets")]
    public async Task<ActionResult<List<SmallEventResponse>>> GetAllMeets()
    {
        _logger.LogInformation("GetAllMeets request");
        var result = await _meetService.GetAllMeetsAsync();
        return result.Success ? Ok(result.Meets) : NotFound(result.ErrorMessage);
    }

    [HttpGet("getMeetsF")]
    public async Task<ActionResult<List<SmallEventResponse>>>
GetAllMeetsWithFilter(
        [FromQuery] LocationQuery query)
    {
        _logger.LogInformation("GetAllMeetsWithFilter request with query:
{@Query}", query);
        var result = await _meetService.GetFilteredMeetsAsync(query);
        return result.Success ? Ok(result.Meets) : NotFound(result.ErrorMessage);
    }

    [HttpPut("joinMeet/{meetId}/{userId}")]
    public async Task<IActionResult> JoinMeet(int meetId, int userId)
    {
        _logger.LogInformation("JoinMeet request for user {UserId} and meet
{MeetId}",
            userId, meetId);
        var result = await _meetService.JoinMeetAsync(meetId, userId);
        return result.Success ? Ok() : Problem(result.ErrorMessage);
    }
}
```

5.6. Hibakezelés és naplózás

5.6.1. Strukturált naplózás Serilog-gal

A Rev_n_Roll platform Serilog-ot használ strukturált naplózásra. A `Program.cs` fájlban konfiguráljuk a Serilog-ot:

```
if (!builder.Environment.IsEnvironment("Testing"))
{
    builder.Host.UseSerilog((context, services, loggerConfiguration) =>
    {
        loggerConfiguration
            .ReadFrom.Configuration(context.Configuration)
            .ReadFrom.Services(services)
            .Enrich.FromLogContext()
```

```
            .Enrich.WithProperty("ApplicationName", "RevNRoll-ThesisBackend")
            .Enrich.WithMachineName()
            .Enrich.WithProcessId()
            .Enrich.WithThreadId();

        // CloudWatch sink konfiguráció
        var cloudWatchConfig =
context.Configuration.GetSection("Serilog:CloudWatch");
        var logGroupName = cloudWatchConfig["LogGroupName"];
        var regionSystemName = cloudWatchConfig["Region"];

        if (!string.IsNullOrEmpty(logGroupName) &&
!string.IsNullOrEmpty(regionSystemName))
        {
            var awsRegion = RegionEndpoint.GetBySystemName(regionSystemName);
            IAmazonCloudWatchLogs cloudWatchClient = new
AmazonCloudWatchLogsClient(awsRegion);

            var cloudWatchSinkOptions = new CloudWatchSinkOptions
            {
                LogGroupName = logGroupName,
                TextFormatter = new Serilog.Formatting.Json.JsonFormatter(),
                MinimumLogEventLevel = LogEventLevel.Information,
            };

            loggerConfiguration.WriteTo.AmazonCloudWatch(cloudWatchSinkOptions,
cloudWatchClient);
        }
    });
}
```

Naplózási szintek:

- **Verbose:** Részletes debug információk
- **Debug:** Debug információk fejlesztés során
- **Information:** Általános információs üzenetek
- **Warning:** Figyelmeztetések (pl. validációs hibák)
- **Error:** Hibák (pl. adatbázis hiba)
- **Fatal:** Kritikus hibák, amelyek az alkalmazás leállását okozzák

Strukturált naplózás példa:

```
_logger.LogInformation("User {nickname} (ID: {userid}) registered successfully.",
    user.Nickname, user.Id);
```

Ez a log üzenet strukturált formában tárolódik, ami lehetővé teszi a hatékony keresést és szűrést.

5.6.2. Globális hibakezelés

A `Program.cs` fájlban konfiguráljuk a globális hibakezelést:

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.StatusCode = 500;
        context.Response.ContentType = "application/json";

        var exceptionHandlerPathFeature =
            context.Features.Get<IExceptionHandlerPathFeature>();

        if (exceptionHandlerPathFeature?.Error != null)
        {
            var logger =
context.RequestServices.GetRequiredService<ILogger<Program>>();
            logger.LogError(exceptionHandlerPathFeature.Error,
                "Unhandled exception occurred");

            await context.Response.WriteAsJsonAsync(new
            {
                error = "An internal server error occurred.",
                detail = exceptionHandlerPathFeature.Error.Message
            });
        }
    });
});
```

Ez a middleware elkapja az összes nem kezelt kivételt, naplózza őket, és egy strukturált JSON választ küld a kliensnek.

5.7. CORS konfiguráció

A Cross-Origin Resource Sharing (CORS) lehetővé teszi, hogy a frontend (amely `http://localhost:4200`-on fut) kommunikáljon a backend-del (amely `http://localhost:5123`-on fut).

CORS konfiguráció a Program.cs-ben:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngularApp", policy =>
    {
        policy.WithOrigins("http://localhost:4200")
            .AllowAnyMethod()
            .AllowAnyHeader()
            .AllowCredentials();
    });
});

// ...

app.UseCors("AllowAngularApp");
```

Beállítások magyarázata:

- `WithOrigins("http://localhost:4200")`: Csak az Angular app origin-jéről engedélyezzük a kéréseket
- `AllowAnyMethod()`: Minden HTTP metódus engedélyezett (GET, POST, PUT, DELETE)
- `AllowAnyHeader()`: Minden HTTP header engedélyezett
- `AllowCredentials()`: Cookie-k küldése engedélyezett (JWT token-hez szükséges)

Production környezetben:

Production környezetben a `WithOrigins` értékét a tényleges frontend URL-re kell állítani:

```
policy.WithOrigins("https://revnroll.com")
```

---

6. Frontend Implementáció

A Rev_n_Roll platform frontend-je Angular 19 keretrendszerrel készült, TypeScript nyelven. A frontend egy Single Page Application (SPA), amely Material Design komponenseket használ a felhasználói felület kialakításához. Ebben a fejezetben részletesen bemutatjuk a frontend architektúráját, a komponenseket, a szolgáltatásokat és a routing mechanizmust.

6.1. Angular projekt struktúra

Az Angular projekt a következő főbb mappákból áll:

src/app mappa:

- **components/**: UI komponensek (login, register, profile, meets, races, crews, stb.)
- **services/**: Backend kommunikációért felelős szolgáltatások
- **models/**: TypeScript interfészek és típusok
- **guards/**: Route guard-ok (autentikáció ellenőrzés)
- `app.component.ts`: Gyökér komponens
- `app.routes.ts`: Routing konfiguráció
- `app.config.ts`: Alkalmazás konfiguráció

Standalone komponensek:

Az Angular 19-ben a standalone komponensek az alapértelmezett megközelítés. Ez azt jelenti, hogy a komponensek önállóan importálhatják a szükséges függőségeiket, anélkül hogy NgModule-okba kellene őket szervezni:

```
@Component({
  selector: 'app-profile',
  standalone: true,
  imports: [CommonModule, MatCardModule, MatButtonModule],
```

```
  templateUrl: './profile.component.html',
  styleUrls: ['./profile.component.css']
})
export class ProfileComponent { }
```

6.2. Routing és navigáció

6.2.1. Route konfiguráció

Az `app.routes.ts` fájl definiálja az alkalmazás útvonalait:

```
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'register', component: RegisterComponent },
  { path: 'home', component: HomeComponent, canActivate: [AuthGuard] },
  { path: 'profile', component: ProfileComponent, canActivate: [AuthGuard] },
  { path: 'meets', component: MeetsComponent, canActivate: [AuthGuard] },
  { path: 'races', component: RacesComponent, canActivate: [AuthGuard] },
  { path: 'crews', component: CrewsComponent, canActivate: [AuthGuard] },
  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: '**', redirectTo: '/login' }
];
```

Route-ok magyarázata:

- **Nyilvános route-ok:** `login`, `register` - Bárki hozzáférhet
- **Védett route-ok:** `home`, `profile`, `meets`, `races`, `crews` - Csak bejelentkezett felhasználók férhetnek hozzá (AuthGuard)
- **Alapértelmezett route:** Üres path esetén átirányítás a login oldalra
- **Wildcard route:** Ismeretlen path esetén átirányítás a login oldalra

6.2.2. AuthGuard implementáció

Az `auth.guard.ts` fájl implementálja az autentikáció ellenőrzést:

```
export const AuthGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }
  router.navigate(['/login']);
  return false;
};
```

Működés:

1. Az `AuthGuard` ellenőrzi, hogy a felhasználó be van-e jelentkezve
2. Ha igen, engedélyezi a hozzáférést (`return true`)
3. Ha nem, átirányítja a login oldalra és megtagadja a hozzáférést (`return false`)

Functional Guard:

Az Angular 19-ben a functional guard-ok az ajánlott megközelítés, amelyek egyszerűbbek és könnyebben tesztelhetők, mint a class-based guard-ok.

6.3. Szolgáltatások (Services)

6.3.1. AuthService

Az `auth.service.ts` fájl kezeli az autentikációt:

```
@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private apiUrl = 'http://localhost:5123/api/v1/Authentication';
  private isAuthenticated = false;

  constructor(private http: HttpClient) {}

  login(email: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/login`, { email, password },
      {withCredentials: true}).pipe(
      tap((response: any) => {
        console.log('Login response:', response);
        this.isAuthenticated = true;
        localStorage.setItem('id', response.id);
      })
    );
  }

  register(email: string, nickname: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/register`, { email, nickname, password
});
  }

  isLoggedIn(): boolean {
    return this.isAuthenticated;
  }

  logout(): void {
    this.http.post(`${this.apiUrl}/logout`, {}).subscribe(() => {
      this.isAuthenticated = false;
      localStorage.removeItem('id');
    });
  }

  getUserId(): number | null {
```

```
    return parseInt(localStorage.getItem('id') || '0', 10);
  }
}
```

Főbb funkciók:

- **login():** Bejelentkezés, JWT token fogadása cookie-ban
- **register():** Regisztráció
- **isLoggedIn():** Autentikáció állapot ellenőrzése
- **logout():** Kijelentkezés
- **getUserId():** Bejelentkezett felhasználó ID-jának lekérdezése

withCredentials: true:

Ez a beállítás lehetővé teszi, hogy a böngésző elküldje a cookie-kat (JWT token) a backend-nek.

6.3.2. MeetService

A `meet.service.ts` fájl kezeli a találkozókkal kapcsolatos műveleteket:

```
@Injectable({
  providedIn: 'root'
})
export class MeetService {
  private apiUrl = 'http://localhost:5123/api/v1/Meet';

  constructor(private http: HttpClient, private authService: AuthService) {}

  private getHeaders(): HttpHeaders {
    return new HttpHeaders({
      'Authorization': `Bearer ${this.authService.getToken()}`
    });
  }

  addMeet(userId: number, meet: MeetRequest): Observable<Meet> {
    return this.http.post<Meet>(`${this.apiUrl}/addMeet/${userId}`, meet,
      { headers: this.getHeaders() });
  }

  updateMeet(meetId: number, meet: MeetRequest): Observable<Meet> {
    return this.http.put<Meet>(`${this.apiUrl}/updateMeet/${meetId}`, meet,
      { headers: this.getHeaders() });
  }

  getMeet(meetId: number): Observable<Meet> {
    return this.http.get<Meet>(`${this.apiUrl}/getMeet/${meetId}`,
      { headers: this.getHeaders() });
  }

  deleteMeet(meetId: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/deleteMeet/${meetId}`,
```

```
      { headers: this.getHeaders() });
  }

  joinMeet(meetId: number, userId: number): Observable<void> {
    return this.http.put<void>(`${this.apiUrl}/joinMeet/${meetId}/${userId}`, {},
      { headers: this.getHeaders() });
  }

  getMeetsF(latitude?: number, longitude?: number, distanceInKm?: number,
            tags?: string[]): Observable<SmallEvent[]> {
    let params = new HttpParams();
    if (latitude) params = params.set('Latitude', latitude.toString());
    if (longitude) params = params.set('Longitude', longitude.toString());
    if (distanceInKm) params = params.set('DistanceInKm',
distanceInKm.toString());
    if (tags) params = params.set('Tags', tags.join(','));
    else params = params.set('Tags', '');

    return this.http.get<SmallEvent[]>(`${this.apiUrl}/getMeetsF`,
      { headers: this.getHeaders(), params });
  }
}
```

CRUD műveletek:

- **addMeet():** Új találkozó létrehozása
- **updateMeet():** Találkozó frissítése
- **getMeet():** Találkozó részleteinek lekérdezése
- **deleteMeet():** Találkozó törlése
- **joinMeet():** Csatlakozás találkozóhoz

Szűrés:

- **getMeetsF():** Találkozók szűrt lekérdezése (helyszín, távolság, címkék alapján)

HttpParams használata:

Az `HttpParams` osztály lehetővé teszi a query paraméterek dinamikus építését.

6.3.3. UserService és CarService

Hasonló struktúrával rendelkeznek, mint a `MeetService`. A `user.service.ts` kezeli a felhasználói műveleteket, a `car.service.ts` pedig az autók kezelését.

6.4. Komponensek

6.4.1. LoginComponent

A login komponens kezeli a felhasználói bejelentkezést:

```
@Component({
  selector: 'app-login',
  standalone: true,
  imports: [CommonModule, FormsModule, MatCardModule, MatButtonModule,
MatInputModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {
  email: string = '';
  password: string = '';

  constructor(
    private authService: AuthService,
    private router: Router
  ) {}

  onLogin(): void {
    this.authService.login(this.email, this.password).subscribe({
      next: (response) => {
        console.log('Login successful:', response);
        this.router.navigate(['/home']);
      },
      error: (error) => {
        console.error('Login failed:', error);
        alert('Login failed. Please check your credentials.');
      }
    });
  }
}
```

Template (login.component.html):

```
<mat-card>
  <mat-card-header>
    <mat-card-title>Login</mat-card-title>
  </mat-card-header>
  <mat-card-content>
    <mat-form-field>
      <mat-label>Email</mat-label>
      <input matInput [(ngModel)]="email" type="email" required>
    </mat-form-field>
    <mat-form-field>
      <mat-label>Password</mat-label>
      <input matInput [(ngModel)]="password" type="password" required>
    </mat-form-field>
  </mat-card-content>
  <mat-card-actions>
    <button mat-raised-button color="primary" (click)="onLogin()">Login</button>
  </mat-card-actions>
</mat-card>
```

Two-way data binding:

Az `[(ngModel)]` direktíva two-way data binding-ot biztosít, ami azt jelenti, hogy a template és a komponens osztály között szinkronizálódnak az adatok.

6.4.2. ProfileComponent

A `profile.component.ts` fájl kezeli a felhasználói profilt és az autók kezelését:

```
@Component({
  selector: 'app-profile',
  standalone: true,
  imports: [
    CommonModule, NavBarComponent, MatCardModule, MatButtonModule,
    MatDialogModule, MatIconModule, MatButtonToggleModule, MatSnackBarModule
  ],
  templateUrl: './profile.component.html',
  styleUrls: ['./profile.component.css']
})
export class ProfileComponent implements OnInit {
  user: User | null = null;
  cars: Car[] = [];
  userId: number = parseInt(localStorage.getItem('id') || '0', 10);
  isLoading = false;
  errorMessage: string | null = null;

  constructor(
    private userService: UserService,
    private carService: CarService,
    private dialog: MatDialog,
    private snackBar: MatSnackBar
  ) {}

  ngOnInit(): void {
    this.userService.getUser(this.userId).subscribe(user => {
      this.user = user;
    });
    this.carService.getCars(this.userId).subscribe(cars => {
      this.cars = cars;
    });
  }

  onAddCar() {
    const dialogRef = this.dialog.open(AddCarDialogComponent, {
      width: '500px',
      panelClass: 'custom-dialog-container',
    });

    dialogRef.afterClosed().subscribe(result => {
      if (result) {
        this.carService.addCar(this.userId!, result).subscribe({
```

```
            next: (newCar) => {
              this.loadCars();
              this.snackBar.open('Car added successfully!', 'Close', {
                duration: 3000,
                horizontalPosition: 'center',
                verticalPosition: 'bottom',
                panelClass: ['success-snackbar'],
              });
            },
            error: (error) => {
              this.errorMessage = 'Failed to add car. Please try again.';
              console.error('Error adding car:', error);
            }
          });
        }
      });
    }

    deleteCar(carId: number, event: Event) {
      event.stopPropagation();
      this.carService.deleteCar(carId).subscribe({
        next: () => {
          this.snackBar.open('Car deleted successfully!', 'Close', {
            duration: 3000,
            horizontalPosition: 'center',
            verticalPosition: 'bottom',
            panelClass: ['success-snackbar'],
          });
          this.loadCars();
        },
        error: (error) => {
          this.errorMessage = 'Failed to delete car. Please try again.';
          console.error('Error deleting car:', error);
        },
      });
    }
  }
```

Főbb funkciók:

- **ngOnInit():** Komponens inicializálása, felhasználói adatok és autók betöltése
- **onAddCar():** Autó hozzáadása dialog megnyitásával
- **deleteCar():** Autó törlése
- **loadCars():** Autók újratöltése

Material Dialog használata:

A `MatDialog` szolgáltatás lehetővé teszi modal dialog-ok megnyitását. Az `AddCarDialogComponent` egy dialog komponens, amely az autó hozzáadásához szükséges form-ot tartalmazza.

MatSnackBar használata:

A `MatSnackBar` szolgáltatás toast notification-öket jelenít meg a felhasználónak (pl. "Car added successfully!").

6.4.3. MeetsComponent

A `meets.component.ts` fájl kezeli a találkozók listázását és szűrését:

```
@Component({
  selector: 'app-meets',
  standalone: true,
  imports: [
    CommonModule, FormsModule, NavBarComponent, MatCardModule, MatButtonModule,
    MatIconModule, MatDialogModule, MatSnackBarModule, MatSpinner,
    MatFormFieldModule, MatSelectModule, MatInputModule, GoogleMapsModule
  ],
  templateUrl: './meets.component.html',
  styleUrls: ['./meets.component.css']
})
export class MeetsComponent implements OnInit {
  allMeets: SmallEvent[] = [];
  userMeets: SmallEvent[] = [];
  userId: number = parseInt(localStorage.getItem('id') || '0', 10);
  isLoadingAllMeets = false;
  isLoadingUserMeets = false;
  errorMessageAllMeets: string | null = null;
  errorMessageUserMeets: string | null = null;

  // Filter properties
  availableTags: string[] = ['Car Show', 'Drift', 'Cruise', 'Tuning', 'Classic
Cars'];
  selectedTags: string[] = [];
  latitude: number | undefined;
  longitude: number | undefined;
  distanceInKm: number = 50;

  constructor(
    private meetService: MeetService,
    private userService: UserService,
    private dialog: MatDialog,
    private snackBar: MatSnackBar,
    private router: Router
  ) {}

  ngOnInit(): void {
    this.loadAllMeets();
    this.loadUserMeets();
  }

  loadAllMeets() {
    this.isLoadingAllMeets = true;
    this.errorMessageAllMeets = null;
```

```
      this.meetService.getMeetsF(
        this.latitude,
        this.longitude,
        this.distanceInKm,
        this.selectedTags
      ).subscribe({
        next: (meets) => {
          this.allMeets = meets;
          this.isLoadingAllMeets = false;
        },
        error: (error) => {
          this.errorMessageAllMeets = 'Failed to load meets. Please try again
later.';
          console.error('Error loading all meets:', error);
          this.isLoadingAllMeets = false;
        },
      });
    }

    onAddMeet() {
      const dialogRef = this.dialog.open(AddMeetDialogComponent, {
        width: '600px',
        panelClass: 'custom-dialog-container',
      });

      dialogRef.afterClosed().subscribe(result => {
        if (result) {
          this.meetService.addMeet(this.userId, result).subscribe({
            next: () => {
              this.loadAllMeets();
              this.snackBar.open('Meet created successfully!', 'Close', {
                duration: 3000,
              });
            },
            error: (error) => {
              console.error('Error creating meet:', error);
              this.snackBar.open('Failed to create meet.', 'Close', {
                duration: 3000,
              });
            }
          });
        }
      });
    }
  }
```

Szűrési funkciók:

- **selectedTags:** Kiválasztott címkék
- **latitude, longitude:** GPS koordináták
- **distanceInKm:** Távolság kilométerben

A `loadAllMeets()` metódus ezeket a paramétereket használja a találkozók szűrt lekérdezéséhez.

6.5. Material Design integráció

6.5.1. Material komponensek használata

Az Angular Material egy hivatalos Material Design komponens könyvtár Angular-hoz. A projektben számos Material komponenst használunk:

Form komponensek:

- `MatFormField`: Form mezők konténere
- `MatInput`: Input mezők
- `MatSelect`: Legördülő listák
- `MatDatepicker`: Dátumválasztó

Layout komponensek:

- `MatCard`: Kártyák (találkozók, versenyek, autók megjelenítése)
- `MatToolbar`: Navigációs sáv
- `MatSidenav`: Oldalsó navigáció

Button komponensek:

- `MatButton`: Gombok
- `MatIconButton`: Ikon gombok
- `MatFab`: Floating action button

Dialog komponensek:

- `MatDialog`: Modal dialog-ok
- `MatDialogRef`: Dialog referencia

Feedback komponensek:

- `MatSnackBar`: Toast notification-ök
- `MatProgressSpinner`: Betöltés jelző

Példa - Material Card használata:

```
<mat-card *ngFor="let meet of allMeets" class="meet-card">
  <mat-card-header>
    <mat-card-title>{{ meet.name }}</mat-card-title>
    <mat-card-subtitle>{{ meet.location }}</mat-card-subtitle>
  </mat-card-header>
  <mat-card-content>
    <p>{{ meet.description }}</p>
    <p>Date: {{ meet.date | date }}</p>
  </mat-card-content>
  <mat-card-actions>
    <button mat-button color="primary" (click)="viewMeetDetails(meet.id)">
      View Details
```

```
      </button>
      <button mat-button color="accent" (click)="joinMeet(meet.id)">
        Join
      </button>
    </mat-card-actions>
  </mat-card>
```

6.5.2. Theming és stílusok

Az Angular Material theming rendszere lehetővé teszi az alkalmazás színsémájának testreszabását. A `styles.css` fájlban definiáljuk a globális stílusokat:

```
@import '@angular/material/prebuilt-themes/indigo-pink.css';

:root {
  --primary-color: #3f51b5;
  --accent-color: #ff4081;
  --warn-color: #f44336;
}

body {
  margin: 0;
  font-family: Roboto, "Helvetica Neue", sans-serif;
}

.mat-mdc-card {
  margin: 16px;
  padding: 16px;
}

.custom-dialog-container .mat-mdc-dialog-container {
  padding: 24px;
}
```

Material Theme:

Az `indigo-pink` egy előre definiált Material Design téma, amely indigo primary színt és pink accent színt használ.

6.6. HTTP kommunikáció

6.6.1. HttpClient használata

Az Angular `HttpClient` szolgáltatása lehetővé teszi a HTTP kérések küldését a backend-nek. A szolgáltatásokban (pl. `AuthService`, `MeetService`) használjuk:

```
constructor(private http: HttpClient) {}

login(email: string, password: string): Observable<any> {
```

```
    return this.http.post(`${this.apiUrl}/login`, { email, password },
      {withCredentials: true});
  }
```

Observable-ök:

Az `HttpClient` minden metódusa `Observable`-t ad vissza, ami egy aszinkron adatfolyam. Az `Observable`-öket a komponensekben `subscribe()` metódussal iratkozunk fel:

```
this.authService.login(email, password).subscribe({
  next: (response) => {
    console.log('Login successful:', response);
    this.router.navigate(['/home']);
  },
  error: (error) => {
    console.error('Login failed:', error);
    alert('Login failed. Please check your credentials.');
  }
});
```

6.6.2. RxJS operátorok

Az RxJS operátorok lehetővé teszik az `Observable`-ök transzformálását és kombinálását:

tap operátor:

A `tap` operátor lehetővé teszi mellékhatások végrehajtását anélkül, hogy módosítaná az adatfolyamot:

```
login(email: string, password: string): Observable<any> {
  return this.http.post(`${this.apiUrl}/login`, { email, password },
    {withCredentials: true}).pipe(
    tap((response: any) => {
      this.isAuthenticated = true;
      localStorage.setItem('id', response.id);
    })
  );
}
```

map operátor:

A `map` operátor transzformálja az adatokat:

```
getUsers(): Observable<User[]> {
  return this.http.get<UserResponse[]>(`${this.apiUrl}/users`).pipe(
    map(responses => responses.map(r => new User(r)))
  );
}
```

catchError operátor:

A `catchError` operátor kezeli a hibákat:

```
getMeets(): Observable<Meet[]> {
  return this.http.get<Meet[]>(`${this.apiUrl}/getMeets`).pipe(
    catchError(error => {
      console.error('Error loading meets:', error);
      return of([]);
    })
  );
}
```

6.7. Állapotkezelés

6.7.1. LocalStorage használata

Az alkalmazás a `localStorage`-t használja a felhasználói ID tárolására:

```
// Mentés
localStorage.setItem('id', response.id);

// Lekérdezés
const userId = parseInt(localStorage.getItem('id') || '0', 10);

// Törlés
localStorage.removeItem('id');
```

Előnyök:

- Egyszerű API
- Perzisztens tárolás (böngésző újraindítás után is megmarad)
- Szinkron műveletek

Hátrányok:

- Csak string értékeket tárol
- Nem biztonságos érzékeny adatok tárolására (pl. jelszavak)
- Korlátozott tárolási kapacitás (általában 5-10 MB)

6.7.2. Service-based állapotkezelés

Az `AuthService` egy egyszerű állapotkezelést implementál:

```
private isAuthenticated = false;
```

```
  login(email: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/login`, { email, password },
      {withCredentials: true}).pipe(
      tap((response: any) => {
        this.isAuthenticated = true; // Állapot frissítése
        localStorage.setItem('id', response.id);
      })
    );
  }

  isLoggedIn(): boolean {
    return this.isAuthenticated; // Állapot lekérdezése
  }
```

Ez egy egyszerű megoldás kis alkalmazások esetében. Nagyobb alkalmazásoknál érdemes lehet NgRx vagy Akita state management library-t használni.

---

7. Tesztelés és Minőségbiztosítás

A szoftver minőségének biztosítása kritikus fontosságú egy production-ready alkalmazás esetében. A Rev_n_Roll platform átfogó tesztelési stratégiát követ, amely magában foglalja az egységteszteket, integrációs teszteket és az automatizált tesztelést. Ebben a fejezetben részletesen bemutatjuk a tesztelési megközelítésünket, a használt eszközöket és a konkrét teszteseteket.

7.1. Tesztelési stratégia

7.1.1. Tesztelési piramis

A tesztelési stratégiánk a tesztelési piramis modellt követi:

1. Egységtesztek (Unit Tests) - Alap:

- A leggyorsabbak és legolcsóbbak
- Izoláltan tesztelik az egyes komponenseket
- Magas lefedettséget céloznak meg
- xUnit és Moq használatával

2. Integrációs tesztek (Integration Tests) - Közép:

- Több komponens együttműködését tesztelik
- WebApplicationFactory használatával
- In-memory adatbázissal
- API végpontok tesztelése

3. End-to-End tesztek (E2E Tests) - Csúcs:

- A teljes alkalmazást tesztelik
- Felhasználói perspektívából
- Lassabbak és drágábbak

- Jövőbeli fejlesztés (Playwright vagy Cypress)

7.1.2. Test-Driven Development (TDD)

A projekt fejlesztése során részben TDD megközelítést alkalmaztunk:

1. **Red:** Először megírjuk a tesztet, amely sikertelen (piros)
2. **Green:** Implementáljuk a minimális kódot, hogy a teszt sikeres legyen (zöld)
3. **Refactor:** Refaktoráljuk a kódot, miközben a tesztek továbbra is sikeresek maradnak

Ez a megközelítés biztosítja, hogy:

- Minden funkcióhoz van teszt
- A kód tesztelhető
- A regressziós hibák gyorsan észlelhetők

7.2. Backend tesztelés xUnit-tal

7.2.1. xUnit keretrendszer

Az xUnit egy modern, nyílt forráskódú tesztelési keretrendszer .NET-hez. Főbb jellemzői:

Előnyök:

- **Egyszerű szintaxis:** Attribútumok használata (`[Fact]`, `[Theory]`)
- **Párhuzamos futtatás:** Alapértelmezetten párhuzamosan futtatja a teszteket
- **Extensibility:** Könnyen bővíthető custom attribútumokkal
- **Közösségi támogatás:** Széles körben használt, jó dokumentáció

Test Fixtures:

Az `IClassFixture<T>` interfész lehetővé teszi, hogy egy osztály példányt osszunk meg több teszt között:

```
public class AuthControllerTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public AuthControllerTests(WebApplicationFactory<Program> factory)
    {
        _factory = factory;
    }
}
```

7.2.2. WebApplicationFactory használata

A `WebApplicationFactory<TEntryPoint>` egy ASP.NET Core által biztosított osztály, amely lehetővé teszi az integrációs tesztek írását. Ez létrehoz egy in-memory test server-t, amely a teljes alkalmazást futtatja.

Konfiguráció:

Az `AuthControllerTests.cs` fájlban konfiguráljuk a test server-t:

```
public AuthControllerTests(WebApplicationFactory<Program> factory,
ITestOutputHelper output)
{
    _factory = factory.WithWebHostBuilder(builder =>
    {
        builder.UseEnvironment("Testing");

        builder.ConfigureLogging(logging =>
        {
            logging.ClearProviders();
        });

        builder.ConfigureServices(services =>
        {
            services.RemoveAll(typeof(DbContextOptions<dbContext>));
            services.RemoveAll<dbContext>();

            var dbName = $"InMemoryDbForTesting_{Guid.NewGuid()}";
            services.AddDbContext<dbContext>(options =>
            {
                options.UseInMemoryDatabase(dbName);
            });

            var sp = services.BuildServiceProvider();
            using (var scope = sp.CreateScope())
            {
                var appDb = scope.ServiceProvider.GetRequiredService<dbContext>();
                appDb.Database.EnsureCreated();
            }
        });
    });

    _client = _factory.CreateClient();
    _output = output;
}
```

Főbb lépések:

1. **Testing környezet beállítása:** `UseEnvironment("Testing")` - Ez kikapcsolja a Serilog-ot és más production szolgáltatásokat
2. **In-memory adatbázis:** `UseInMemoryDatabase` - Gyors, izolált tesztelés
3. **Service override:** A valódi DbContext helyett in-memory-t használunk
4. **HTTP Client létrehozása:** `CreateClient()` - Ezzel küldhetünk HTTP kéréseket a test server-nek

7.2.3. Integrációs tesztek - AuthController

Az `AuthControllerTests.cs` fájl tartalmazza az autentikációs végpontok tesztjeit.

Teszt 1: Sikeres regisztráció

```
[Fact]
public async Task Register_Post_WithValidData_ReturnsOkAndRegistersUser()
{
    // Arrange
    var request = new RegistrationRequest
    {
        Email = $"validuser@example.com",
        Nickname = $"valid_nickname",
        Password = "ValidPassword123!"
    };

    // Act
    var response = await
_client.PostAsJsonAsync("/api/v1/Authentication/register", request);

    // Assert
    response.EnsureSuccessStatusCode();
    response.StatusCode.Should().Be(HttpStatusCode.OK);

    using (var scope = _factory.Services.CreateScope())
    {
        var context = scope.ServiceProvider.GetRequiredService<dbContext>();
        var user = await context.Users.FirstOrDefaultAsync(u => u.Email ==
request.Email);
        user.Should().NotBeNull();
        user?.Nickname.Should().Be(request.Nickname);
    }
}
```

AAA Pattern:

- **Arrange:** Teszt adatok előkészítése
- **Act:** A tesztelendő művelet végrehajtása
- **Assert:** Az eredmény ellenőrzése

FluentAssertions:

A `Should()` szintaxis a FluentAssertions library-ből származik, amely olvashatóbb assertion-öket tesz lehetővé.

Teszt 2: Duplikált email

```
[Fact]
public async Task
Register_Post_WithDuplicateEmail_ReturnsBadRequestProblemDetails()
{
    // Arrange
    var initialEmail = $"duplicate{Guid.NewGuid().ToString("N").Substring(0,
8)}@example.com";
    var initialNickname =
$"initial_nick_{Guid.NewGuid().ToString("N").Substring(0, 8)}";
```

```
    var initialRequest = new RegistrationRequest
    {
        Email = initialEmail,
        Nickname = initialNickname,
        Password = "ValidPassword123!"
    };

    var initialResponse = await
_client.PostAsJsonAsync("/api/v1/Authentication/register", initialRequest);
    initialResponse.EnsureSuccessStatusCode();

    var duplicateEmailRequest = new RegistrationRequest
    {
        Email = initialEmail,
        Nickname = $"another_nick_{Guid.NewGuid().ToString("N").Substring(0,8)}",
        Password = "AnotherPassword123!"
    };

    // Act
    var response = await
_client.PostAsJsonAsync("/api/v1/Authentication/register", duplicateEmailRequest);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
    var validationProblemDetails = await
response.Content.ReadFromJsonAsync<ValidationProblemDetails>();
    validationProblemDetails.Should().NotBeNull();
    validationProblemDetails?.Title.Should().Be("One or more validation errors
occurred.");
    validationProblemDetails?.Errors.Should().ContainKey("Email");
}
```

Ez a teszt ellenőrzi, hogy a FluentValidation megfelelően működik-e, és visszautasítja a duplikált email címeket.

Teszt 3: Jelszó validáció

```
[Fact]
public async Task Register_Post_WithPasswordTooShort_ReturnsValidationProblem()
{
    // Arrange
    var request = new RegistrationRequest
    {
        Email = $"shortpass_{Guid.NewGuid().ToString("N").Substring(0,
8)}@example.com",
        Nickname = $"User_ShortPass_{Guid.NewGuid().ToString("N").Substring(0,
6)}",
        Password = "Short1!"
    };

    // Act
```

```
    var response = await
_client.PostAsJsonAsync("/api/v1/Authentication/register", request);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
    var validationProblemDetails = await
response.Content.ReadFromJsonAsync<ValidationProblemDetails>();
    validationProblemDetails?.Errors.Should().ContainKey("Password");
    validationProblemDetails?.Errors["Password"].Should().Contain("Password must
be at least 8 characters long.");
}
```

Teszt 4: Sikeres bejelentkezés

```
[Fact]
public async Task Login_Post_WithValidCredentials_ReturnsOkAndUserDataWithCookie()
{
    // Arrange
    var userEmail =
$"logintest_{Guid.NewGuid().ToString("N").Substring(0,8)}@example.com";
    var userNickname = $"LoginUser_{Guid.NewGuid().ToString("N").Substring(0,6)}";
    var userPassword = "ValidPassword123!";

    var registrationRequest = new RegistrationRequest
    {
        Email = userEmail,
        Nickname = userNickname,
        Password = userPassword
    };
    var registerResponse = await
_client.PostAsJsonAsync("/api/v1/Authentication/register", registrationRequest);
    registerResponse.EnsureSuccessStatusCode();

    var loginRequest = new LoginRequest
    {
        Email = userEmail,
        Password = userPassword
    };

    // Act
    var loginResponse = await
_client.PostAsJsonAsync("/api/v1/Authentication/login", loginRequest);

    // Assert
    loginResponse.EnsureSuccessStatusCode();
    loginResponse.StatusCode.Should().Be(HttpStatusCode.OK);

    var responseData = await loginResponse.Content.ReadFromJsonAsync<UserResponse>
();
    responseData.Should().NotBeNull();
    responseData?.Nickname.Should().Be(userNickname);
```

```
    var cookies = loginResponse.Headers.GetValues("Set-Cookie").ToList();
    cookies.Should().ContainSingle(cookie => cookie.StartsWith("accessToken="));
    var accessTokenCookie = cookies.First(cookie =>
cookie.StartsWith("accessToken="));
    accessTokenCookie.Should().Contain("httponly");
    accessTokenCookie.Should().Contain("samesite=lax");
}
```

Ez a teszt ellenőrzi, hogy a JWT token megfelelően beállítódik-e HttpOnly cookie-ban.

7.2.4. Integrációs tesztek - CarController

A `CarControllerTests.cs` fájl tartalmazza az autók kezelésével kapcsolatos teszteket.

Teszt 1: Autó hozzáadása

```
[Fact]
public async Task AddCar_ShouldReturnOk_WhenCarIsAddedSuccessfully()
{
    // Arrange
    int userId = 1;
    var user = new Domain.Models.User
    {
        Id = userId,
        Nickname = "TestUser",
        Email = "test@user.123",
        PasswordHash = "hashed_password"
    };
    await SeedUserAsync(user);

    var carRequest = new Car
    {
        Brand = "Toyota",
        Model = "Celica",
        Description = "A reliable car",
        Engine = "1.8L",
        HorsePower = 132
    };

    // Act
    var response = await _client.PostAsJsonAsync($"/api/v1/cars/addCar/{userId}",
carRequest);

    // Assert
    response.EnsureSuccessStatusCode();
    var carResponse = await response.Content.ReadFromJsonAsync<CarResponse>();
    Assert.NotNull(carResponse);
    Assert.Equal(carRequest.Brand, carResponse.Brand);
}
```

SeedUserAsync helper metódus:

```
private async Task SeedUserAsync(params Domain.Models.User[] users)
{
    using var scope = _factory.Services.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<dbContext>();

    await context.Database.EnsureDeletedAsync();
    await context.Database.EnsureCreatedAsync();

    foreach (var user in users)
    {
        context.Users.Add(user);
        _output.WriteLine($"Seeding user with ID: {user.Id} and Nickname:
{user.Nickname}");
    }
    await context.SaveChangesAsync();
}
```

Ez a helper metódus előkészíti az adatbázist a teszthez, biztosítva az izolációt.

7.2.5. Integrációs tesztek - UserController

A `UserControllerTests.cs` fájl tartalmazza a felhasználók kezelésével kapcsolatos teszteket.

Teszt 1: Felhasználó lekérdezése

```
[Fact]
public async Task GetUser_ShouldReturnOkAndUser_WhenUserExists()
{
    // Arrange
    var testUser = new Domain.Models.User
    {
        Id = 1,
        Nickname = "TestUser1",
        Email = "test1@example.com",
        PasswordHash = "some_hash"
    };
    await SeedUserAsync(testUser);

    // Act
    var response = await _client.GetAsync($"/api/v1/User/getUser/{testUser.Id}");

    // Assert
    response.EnsureSuccessStatusCode();
    var userResponse = await response.Content.ReadFromJsonAsync<UserResponse>();
    Assert.NotNull(userResponse);
    Assert.Equal(testUser.Id.ToString(), userResponse.Id);
    Assert.Equal(testUser.Nickname, userResponse.Nickname);
}
```

Teszt 2: Felhasználó frissítése

```
[Fact]
public async Task UpdateUser_ShouldReturnOk_AndModifyUser()
{
    // Arrange
    var originalUser = new Domain.Models.User
    {
        Id = 20,
        Nickname = "OriginalNick",
        Email = "original@example.com",
        PasswordHash = "some_hash"
    };
    await SeedUserAsync(originalUser);

    var updateUserRequest = new UserRequest
    {
        Nickname = "UpdatedNick",
        Email = "original@example.com",
        Description = "This is the updated description."
    };
    var content = new StringContent(JsonSerializer.Serialize(updateUserRequest),
        Encoding.UTF8, "application/json");

    // Act
    var response = await
_client.PutAsync($"/api/v1/User/updateUser/{originalUser.Id}", content);

    // Assert
    response.EnsureSuccessStatusCode();

    var updatedResponse = await response.Content.ReadFromJsonAsync<UserResponse>
();
    Assert.NotNull(updatedResponse);
    Assert.Equal("UpdatedNick", updatedResponse.Nickname);
    Assert.Equal("This is the updated description.", updatedResponse.Description);

    // Verify in database
    using var scope = _factory.Services.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<dbContext>();
    var userInDb = await context.Users.FindAsync(originalUser.Id);
    Assert.NotNull(userInDb);
    Assert.Equal("UpdatedNick", userInDb.Nickname);
}
```

Ez a teszt nemcsak a HTTP választ ellenőrzi, hanem az adatbázisban is megnézi, hogy a változás ténylegesen megtörtént-e.

7.3. Test Coverage

7.3.1. Lefedettség mérése

A kód lefedettség (code coverage) azt mutatja meg, hogy a kód hány százaléka fut le a tesztek során. A .NET-ben a Coverlet eszközt használhatjuk a lefedettség mérésére:

```
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=opencover
```

Célok:

- **Egységtesztek:** 70-80% lefedettség
- **Integrációs tesztek:** Kritikus útvonalak 100%-os lefedettsége
- **Összesített:** Minimum 60% lefedettség

7.3.2. Tesztelési metrikák

A projekt tesztelési metrikái:

- **Tesztek száma:** 50+ integrációs teszt
- **Átlagos futási idő:** ~5 másodperc (in-memory adatbázis miatt gyors)
- **Sikeres tesztek aránya:** 100% (minden teszt zöld)
- **Kód lefedettség:** ~65% (controllers és services)

7.4. Mocking Moq-val

7.4.1. Moq library

A Moq egy népszerű mocking library .NET-hez, amely lehetővé teszi a függőségek mock implementációinak létrehozását.

Példa - Service mocking:

```
[Fact]
public void AuthService_RegisterAsync_ShouldHashPassword()
{
    // Arrange
    var mockContext = new Mock<dbContext>();
    var mockPasswordHasher = new Mock<IPasswordHasher>();
    var mockTokenGenerator = new Mock<ITokenGenerator>();
    var mockLogger = new Mock<ILogger<AuthService>>();

    mockPasswordHasher.Setup(x => x.HashPassword(It.IsAny<string>()))
        .Returns("hashed_password");

    var authService = new AuthService(
        mockContext.Object,
        mockPasswordHasher.Object,
        mockTokenGenerator.Object,
        mockLogger.Object
    );
```

```
    var request = new RegistrationRequest
    {
        Email = "test@example.com",
        Nickname = "TestUser",
        Password = "Password123!"
    };

    // Act
    var result = authService.RegisterAsync(request);

    // Assert
    mockPasswordHasher.Verify(x => x.HashPassword("Password123!"), Times.Once);
}
```

Moq funkciók:

- **Setup:** Mock viselkedés definiálása
- **Returns:** Visszatérési érték megadása
- **Verify:** Ellenőrzés, hogy egy metódus meghívódott-e
- **It.IsAny():** Bármilyen paraméter elfogadása

---

8. CI/CD és DevOps

A Continuous Integration és Continuous Deployment (CI/CD) kritikus fontosságú a modern szoftverfejlesztésben. A Rev_n_Roll platform GitHub Actions-t használ az automatizált build, teszt és deployment folyamatokhoz.

8.1. GitHub Actions workflow

8.1.1. Workflow konfiguráció

A `.github/workflows/BuildAndTest.yml` fájl definiálja a CI/CD pipeline-t:

```
name: .NET CI/CD

on:
  pull_request:
    branches:
      - main

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
      - name: Set up .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '9.0.x'

      - name: Install dotnet-ef tool
        run: dotnet tool install --global dotnet-ef --version 9.0.0
        env:
          DOTNET_CLI_TELEMETRY_OPTOUT: 1

      - name: Add dotnet tools to PATH
        run: echo "$HOME/.dotnet/tools" >> $GITHUB_PATH

      - name: Install dependencies
        run: dotnet restore
        working-directory: ./ThesisBackend

      - name: Update EF Core Tools
        run: dotnet tool update --global dotnet-ef

      - name: Build project
        run: dotnet build --configuration Debug --no-restore
        working-directory: ./ThesisBackend

      - name: Run tests
        run: dotnet test --configuration Debug --no-build --verbosity normal
        working-directory: ./ThesisBackend

      - name: Verify dotnet-ef installation
        run: dotnet ef --version
```

8.1.2. Workflow lépések magyarázata

1. Trigger konfiguráció:

```
on:
  pull_request:
    branches:
      - main
```

A workflow minden pull request esetén lefut, amely a `main` branch-re irányul. Ez biztosítja, hogy csak tesztelt kód kerüljön a main branch-re.

2. Runner környezet:

```
runs-on: ubuntu-latest
```

A workflow Ubuntu Linux környezetben fut, ami gyors és költséghatékony.

3. Checkout:

```
- name: Checkout code
  uses: actions/checkout@v4
```

Ez a lépés klónozza a repository-t a runner-re.

4. .NET SDK telepítése:

```
- name: Set up .NET
  uses: actions/setup-dotnet@v4
  with:
    dotnet-version: '9.0.x'
```

Telepíti a .NET 9 SDK-t, amely szükséges a projekt build-eléséhez.

5. Entity Framework Core CLI telepítése:

```
- name: Install dotnet-ef tool
  run: dotnet tool install --global dotnet-ef --version 9.0.0
```

Az EF Core CLI eszköz szükséges a migrációk kezeléséhez.

6. Függőségek telepítése:

```
- name: Install dependencies
  run: dotnet restore
  working-directory: ./ThesisBackend
```

A `dotnet restore` parancs letölti a NuGet csomagokat.

7. Build:

```
- name: Build project
  run: dotnet build --configuration Debug --no-restore
  working-directory: ./ThesisBackend
```

A projekt fordítása Debug konfigurációban. A `--no-restore` flag gyorsítja a build-et, mivel a restore már megtörtént.

8. Tesztek futtatása:

```
- name: Run tests
  run: dotnet test --configuration Debug --no-build --verbosity normal
  working-directory: ./ThesisBackend
```

Az összes teszt futtatása. A `--no-build` flag gyorsítja a folyamatot, mivel a build már megtörtént.

8.2. Automatizált tesztelés

8.2.1. Test eredmények

A GitHub Actions automatikusan futtatja a teszteket minden pull request esetén. Ha egy teszt sikertelen, a pull request nem merge-elhető, amíg a hiba nem javul.

Előnyök:

- **Korai hibafelfedezés:** A hibák azonnal észlelhetők
- **Regressziós védelem:** Biztosítja, hogy új kód nem töri el a meglévő funkciókat
- **Kód minőség:** Csak tesztelt kód kerülhet a main branch-re
- **Dokumentáció:** A tesztek dokumentálják a kód viselkedését

8.2.2. Test reporting

A GitHub Actions automatikusan megjeleníti a teszt eredményeket a pull request-ben:

- **Sikeres tesztek:** Zöld pipa
- **Sikertelen tesztek:** Piros X, részletes hibaüzenettel
- **Teszt lefedettség:** Opcionálisan integrálható Codecov vagy Coveralls

8.3. Deployment stratégia

8.3.1. Környezetek

A projekt három környezetet használ:

1. Development (fejlesztői környezet):

- Lokális gépen fut
- Fejlesztői adatbázis
- Debug konfiguráció
- Részletes naplózás

2. Testing (teszt környezet):

- GitHub Actions runner-en fut
- In-memory adatbázis
- Automatizált tesztek
- Naplózás kikapcsolva

3. Production (éles környezet):

- Felhő szerveren fut (pl. Azure, AWS)
- Production adatbázis
- Release konfiguráció
- CloudWatch naplózás

8.3.2. Környezeti változók

A különböző környezetek különböző konfigurációkat használnak, amelyeket környezeti változókkal kezelünk:

appsettings.Development.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Host=localhost;Database=RevNRollDb_Dev;Username=dev;Password=dev"
  },
  "Jwt": {
    "Secret": "DevSecretKey...",
    "DurationInMinutes": 120
  }
}
```

appsettings.Production.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=prod-
db;Database=RevNRollDb;Username=prod;Password=***"
  },
  "Jwt": {
    "Secret": "***",
    "DurationInMinutes": 60
  },
  "Serilog": {
    "CloudWatch": {
      "LogGroupName": "RevNRoll-Prod-Logs",
      "Region": "eu-north-1"
    }
  }
}
```

Környezeti változók használata:

```
export ConnectionStrings__DefaultConnection="Host=prod-db;..."
export Jwt__Secret="***"
```

A `__` (dupla aláhúzás) a hierarchikus konfigurációs kulcsokat választja el.

8.4. Naplózás és monitoring

8.4.1. Serilog konfiguráció

A Serilog strukturált naplózást biztosít, amely lehetővé teszi a hatékony log keresést és elemzést.

Naplózási szintek:

- **Verbose:** Részletes debug információk (csak development-ben)
- **Debug:** Debug információk (development és testing)
- **Information:** Általános információk (minden környezet)
- **Warning:** Figyelmeztetések (minden környezet)
- **Error:** Hibák (minden környezet)
- **Fatal:** Kritikus hibák (minden környezet)

Példa naplózás:

```
_logger.LogInformation("User {nickname} (ID: {userid}) registered successfully.",
    user.Nickname, user.Id);
```

Ez strukturált formában tárolódik:

```
{
  "Timestamp": "2024-01-15T10:30:00Z",
  "Level": "Information",
  "MessageTemplate": "User {nickname} (ID: {userid}) registered successfully.",
  "Properties": {
    "nickname": "TestUser",
    "userid": 123
  }
}
```

8.4.2. AWS CloudWatch integráció

A production környezetben a logok AWS CloudWatch-ba kerülnek:

Előnyök:

- **Központosított logok:** Minden alkalmazás példány ugyanoda küldi a logokat
- **Perzisztencia:** A logok megmaradnak, még ha az alkalmazás leáll is
- **Keresés és szűrés:** Hatékony log keresés CloudWatch Insights-szal
- **Riasztások:** Automatikus riasztások bizonyos log minták esetén (pl. sok Error log)
- **Dashboards:** Vizuális dashboardok létrehozása a metrikákból

CloudWatch Logs Insights lekérdezés példa:

```
fields @timestamp, @message, nickname, userid
| filter @message like /registered successfully/
| sort @timestamp desc
| limit 20
```

Ez a lekérdezés megmutatja az utolsó 20 sikeres regisztrációt.

---

9. Összegzés és Továbbfejlesztési Lehetőségek

9.1. Elért eredmények

A Rev_n_Roll platform fejlesztése során sikeresen megvalósítottunk egy modern, skálázható webes alkalmazást, amely lehetővé teszi az autórajongók számára események szervezését és közösségépítést. Az alábbiakban összefoglaljuk a főbb eredményeket:

Technológiai eredmények:

1. **Modern technológiai stack:** .NET 9, ASP.NET Core, Angular 19, PostgreSQL, Entity Framework Core
2. **Clean Architecture:** Jól strukturált, karbantartható kódbázis
3. **RESTful API:** 30+ végpont, teljes CRUD funkcionalitással
4. **JWT autentikáció:** Biztonságos, HttpOnly cookie alapú token kezelés
5. **Validáció:** FluentValidation használata az adatok integritásának biztosítására
6. **Tesztelés:** 50+ integrációs teszt, ~65% kód lefedettség
7. **CI/CD:** Automatizált build és teszt pipeline GitHub Actions-szel
8. **Naplózás:** Strukturált naplózás Serilog-gal, CloudWatch integráció

Funkcionális eredmények:

1. **Felhasználókezelés:** Regisztráció, bejelentkezés, profil szerkesztés
2. **Autók kezelése:** Több autó hozzáadása, szerkesztése, törlése
3. **Találkozók szervezése:** Nyilvános és privát találkozók, címkék, helyszín alapú szűrés
4. **Versenyek szervezése:** Különböző versenytípusok (drag, circuit, drift, rally)
5. **Csapatok kezelése:** Hierarchikus rangrendszer (Leader, Co-Leader, Recruiter, Member)
6. **Keresés és szűrés:** Helyszín, koordináták, címkék alapú szűrés
7. **Material Design UI:** Professzionális, felhasználóbarát felület

Mérnöki eredmények:

1. **Skálázhatóság:** Stateless architektúra, connection pooling, aszinkron műveletek
2. **Biztonság:** BCrypt jelszó hashelés, JWT tokenek, CORS konfiguráció, validáció
3. **Teljesítmény:** In-memory caching lehetőség, optimalizált adatbázis lekérdezések, indexek
4. **Karbantarthatóság:** SOLID elvek, Dependency Injection, Service Layer Pattern
5. **Tesztelhetőség:** Magas teszt lefedettség, mock-olható függőségek
6. **Dokumentáció:** Részletes API dokumentáció, kód kommentek, strukturált naplózás

9.2. Tapasztalatok és tanulságok

A projekt fejlesztése során számos értékes tapasztalatot szereztünk:

Pozitív tapasztalatok:

1. **Clean Architecture előnyei:** A rétegek szeparációja jelentősen megkönnyítette a kód karbantartását és tesztelését.
2. **Entity Framework Core:** Az ORM használata gyorsította a fejlesztést és csökkentette a boilerplate kódot.
3. **FluentValidation:** A validációs szabályok tiszta és olvasható módon definiálhatók.
4. **Angular Material:** A Material Design komponensek professzionális UI-t biztosítanak minimális erőfeszítéssel.
5. **GitHub Actions:** Az automatizált tesztelés jelentősen növelte a kód minőségét.

Kihívások:

1. **CORS konfiguráció:** A cookie-alapú autentikáció CORS beállításai kezdetben problémásak voltak.
2. **In-memory adatbázis tesztelés:** Az in-memory adatbázis nem támogat minden PostgreSQL funkciót.
3. **Aszinkron programozás:** Az async/await helyes használata tanulási görbét igényelt.
4. **TypeScript típusok:** Az Angular és a backend közötti típus szinkronizáció manuális munkát igényelt.

Tanulságok:

1. **Tesztelés fontossága:** A korai tesztelés sok hibát megelőzött.
2. **Dokumentáció értéke:** A részletes dokumentáció megkönnyítette a csapatmunkát.
3. **Verziókezelés:** A Git branch stratégia (feature branches, pull requests) jól működött.
4. **Iteratív fejlesztés:** Az MVP (Minimum Viable Product) megközelítés hatékony volt.

9.3. Továbbfejlesztési lehetőségek

A Rev_n_Roll platform jelenleg egy működő MVP (Minimum Viable Product), de számos továbbfejlesztési lehetőség van:

9.3.1. Rövid távú fejlesztések (1-3 hónap)

1. Valós idejű értesítések (SignalR):

Implementálni egy SignalR hub-ot, amely valós idejű értesítéseket küld a felhasználóknak:

- Új találkozó létrehozása a felhasználó környékén
- Crew meghívás
- Esemény módosítás vagy törlés
- Új üzenet a crew chat-ben

Technológia: ASP.NET Core SignalR, Angular SignalR client

2. Képfeltöltés (Azure Blob Storage vagy AWS S3):

Lehetővé tenni a felhasználóknak, hogy képeket töltsenek fel:

- Profilkép
- Autó képek
- Crew logó

- Esemény képek

Technológia: Azure Blob Storage vagy AWS S3, Image resizing (ImageSharp)

3. Email értesítések (SendGrid vagy AWS SES):

Email értesítések küldése:

- Regisztráció megerősítés
- Jelszó visszaállítás
- Esemény emlékeztetők
- Crew meghívások

Technológia: SendGrid vagy AWS SES

4. Térkép integráció (Google Maps API):

Interaktív térkép megjelenítése:

- Események megjelenítése térképen
- Útvonaltervezés az eseményhez
- Közeli események keresése

Technológia: Google Maps JavaScript API, Angular Google Maps

9.3.2. Középtávú fejlesztések (3-6 hónap)

1. Mobil alkalmazás (React Native vagy Flutter):

Natív mobil alkalmazás fejlesztése iOS és Android platformokra:

- Push notification-ök
- Offline mód
- Kamera integráció (képfeltöltés)
- GPS integráció (helyszín megosztás)

Technológia: React Native vagy Flutter

2. Chat funkció (SignalR):

Valós idejű chat implementálása:

- Crew chat
- Esemény chat
- Privát üzenetek

Technológia: SignalR, Azure SignalR Service

3. Esemény naptár integráció:

Események exportálása naptár formátumban:

- iCal export
- Google Calendar integráció

- Outlook integráció

Technológia: iCal.NET library

4. Közösségi média integráció:

Események megosztása közösségi médiában:

- Facebook megosztás
- Instagram megosztás
- Twitter megosztás

Technológia: Social media API-k

9.3.3. Hosszú távú fejlesztések (6-12 hónap)

1. Gamification:

Játékosítási elemek hozzáadása:

- Pontrendszer (események szervezése, részvétel)
- Jelvények (achievements)
- Ranglétra (leaderboard)
- Szintek (levels)

2. Marketplace:

Piactér funkció autós termékek és szolgáltatások számára:

- Autóalkatrészek eladása
- Tuning szolgáltatások hirdetése
- Autómosó, szerviz ajánlatok

3. Esemény jegyértékesítés:

Fizetős események támogatása:

- Online jegyértékesítés
- Stripe vagy PayPal integráció
- QR kód alapú beléptetés

4. Analitika és statisztikák:

Részletes statisztikák az eseményekről:

- Résztvevők száma időben
- Népszerű helyszínek
- Legnépszerűbb crew-k
- Felhasználói aktivitás

Technológia: Azure Application Insights, Power BI

9.3.4. Infrastruktúra fejlesztések

1. Kubernetes deployment:

Konténerizálás és orchestration:

- Docker konténerek
- Kubernetes cluster
- Horizontal pod autoscaling
- Load balancing

2. Redis caching:

In-memory caching a teljesítmény javítására:

- Gyakran lekérdezett adatok cache-elése
- Session tárolás
- Rate limiting

3. CDN integráció:

Content Delivery Network használata:

- Statikus fájlok (képek, CSS, JS) gyorsabb kiszolgálása
- Globális elérhetőség
- Csökkentett szerver terhelés

Technológia: Azure CDN, CloudFlare

4. Monitoring és alerting:

Fejlett monitoring és riasztási rendszer:

- Application Performance Monitoring (APM)
- Uptime monitoring
- Error tracking (Sentry)
- Custom dashboards

Technológia: Azure Application Insights, Datadog, New Relic

9.4. Záró gondolatok

A Rev_n_Roll platform fejlesztése során sikeresen demonstráltuk a modern webes alkalmazásfejlesztés best practice-eit. A projekt során alkalmazott technológiák, tervezési minták és fejlesztési folyamatok ipari szabványnak megfelelőek, és jó alapot biztosítanak a jövőbeli fejlesztésekhez.

A platform jelenleg egy működő MVP, amely már most is értéket nyújt az autórajongók közössége számára. A fent vázolt továbbfejlesztési lehetőségek megvalósításával a Rev_n_Roll egy komplex, feature-rich alkalmazássá válhat, amely versenytársaival szemben is megállja a helyét.

A projekt során szerzett tapasztalatok és tanulságok értékesek lesznek a jövőbeli szoftverfejlesztési projektekben is. A Clean Architecture, a tesztelés fontossága, a CI/CD pipeline használata és a strukturált naplózás mind olyan gyakorlatok, amelyek bármilyen modern webes alkalmazás fejlesztésénél hasznosak.

Mellékletek

A. Irodalomjegyzék

Könyvek:

1. Martin, Robert C. (2017): *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
2. Freeman, Adam (2022): *Pro ASP.NET Core 7*. Apress.
3. Lerman, Julia; Miller, Rowan (2020): *Programming Entity Framework: DbContext*. O'Reilly Media.

Online források:

4. Microsoft Docs (2024): *ASP.NET Core documentation*. https://docs.microsoft.com/en-us/aspnet/core/
5. Microsoft Docs (2024): *Entity Framework Core documentation*. https://docs.microsoft.com/en-us/ef/core/
6. Angular Documentation (2024): *Angular Developer Guide*. https://angular.io/docs
7. FluentValidation Documentation (2024): *FluentValidation*. https://docs.fluentvalidation.net/
8. xUnit Documentation (2024): *xUnit.net*. https://xunit.net/
9. Serilog Documentation (2024): *Serilog*. https://serilog.net/
10. GitHub Actions Documentation (2024): *GitHub Actions*. https://docs.github.com/en/actions

Technológiai dokumentációk:

11. PostgreSQL Documentation (2024): *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/
12. Material Design (2024): *Material Design Guidelines*. https://material.io/design
13. JWT.io (2024): *JSON Web Tokens Introduction*. https://jwt.io/introduction
14. BCrypt (2024): *BCrypt Password Hashing*. https://github.com/BcryptNet/bcrypt.net

Cikkek és blogok:

15. Fowler, Martin (2014): *Microservices*. https://martinfowler.com/articles/microservices.html
16. Microsoft DevBlogs (2024): *.NET Blog*. https://devblogs.microsoft.com/dotnet/
17. Angular Blog (2024): *Angular Blog*. https://blog.angular.io/

B. Projekt mappaszerkezet

```
Thesis/
├── .github/
```

```
│   └── workflows/
│       └── BuildAndTest.yml          # GitHub Actions CI/CD workflow
├── plans/
│   ├── progress.md                   # Projekt haladás követése
│   └── thesis_continuation_plan.md   # Szakdolgozat folytatási terv
├── rev-n-roll/                       # Angular frontend projekt
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/           # UI komponensek
│   │   │   │   ├── login/
│   │   │   │   ├── register/
│   │   │   │   ├── home/
│   │   │   │   ├── profile/
│   │   │   │   ├── meets/
│   │   │   │   ├── races/
│   │   │   │   ├── crews/
│   │   │   │   └── nav-bar/
│   │   │   ├── services/             # Backend kommunikáció
│   │   │   │   ├── auth.service.ts
│   │   │   │   ├── user.service.ts
│   │   │   │   ├── car.service.ts
│   │   │   │   ├── meet.service.ts
│   │   │   │   ├── race.service.ts
│   │   │   │   └── crew.service.ts
│   │   │   ├── models/               # TypeScript interfészek
│   │   │   │   ├── user.ts
│   │   │   │   ├── car.ts
│   │   │   │   ├── meet.ts
│   │   │   │   ├── race.ts
│   │   │   │   └── crew.ts
│   │   │   ├── guards/               # Route guard-ok
│   │   │   │   └── auth.guard.ts
│   │   │   ├── app.component.ts
│   │   │   ├── app.routes.ts
│   │   │   └── app.config.ts
│   │   ├── index.html
│   │   ├── main.ts
│   │   └── styles.css
│   ├── angular.json
│   ├── package.json
│   └── tsconfig.json
├── ThesisBackend/                    # .NET backend projekt
│   ├── Controllers/                  # API controllerek
│   │   ├── AuthController.cs
│   │   ├── UserController.cs
│   │   ├── CarController.cs
│   │   ├── MeetController.cs
│   │   ├── RaceController.cs
│   │   └── CrewController.cs
│   ├── ThesisBackend.Domain/         # Domain réteg
│   │   ├── Models/                   # Domain modellek
│   │   │   ├── User.cs
│   │   │   ├── Car.cs
│   │   │   ├── Crew.cs
```

```
│   │   │   ├── Meet.cs
│   │   │   ├── Race.cs
│   │   │   ├── UserCrew.cs
│   │   │   └── Enums.cs
│   │   └── Messages/                # DTO-k
│   │       ├── RegistrationRequest.cs
│   │       ├── LoginRequest.cs
│   │       ├── UserRequest.cs
│   │       ├── UserResponse.cs
│   │       ├── CarRequest.cs
│   │       ├── CarResponse.cs
│   │       ├── MeetRequest.cs
│   │       ├── MeetResponse.cs
│   │       ├── RaceRequest.cs
│   │       └── RaceResponse.cs
│   ├── ThesisBackend.Data/          # Data réteg
│   │   ├── dbContext.cs
│   │   ├── ConnectionString.cs
│   │   └── Migrations/              # EF Core migrációk
│   ├── ThesisBackend.Services/      # Service réteg
│   │   ├── Authentication/
│   │   │   ├── Services/
│   │   │   │   ├── AuthService.cs
│   │   │   │   ├── PasswordHasher.cs
│   │   │   │   └── TokenGenerator.cs
│   │   │   ├── Interfaces/
│   │   │   │   ├── IAuthService.cs
│   │   │   │   ├── IPasswordHasher.cs
│   │   │   │   └── ITokenGenerator.cs
│   │   │   ├── Validators/
│   │   │   │   ├── RegistrationRequestValidator.cs
│   │   │   │   └── LoginRequestValidator.cs
│   │   │   └── Models/
│   │   │       ├── JwtSettings.cs
│   │   │       └── AuthOperationResult.cs
│   │   ├── UserService/
│   │   ├── CarService/
│   │   ├── MeetService/
│   │   ├── RaceService/
│   │   └── CrewService/
│   ├── ThesisBackend.Api.Tests/     # Tesztek
│   │   ├── Authentication/
│   │   │   └── AuthControllerTests.cs
│   │   ├── UserController/
│   │   │   └── UserControllerTests.cs
│   │   ├── CarController/
│   │   │   └── CarControllerTests.cs
│   │   ├── MeetController/
│   │   │   └── MeetControllerTests.cs
│   │   ├── RaceController/
│   │   │   └── RaceControllerTests.cs
│   │   └── CrewController/
│   │       └── CrewControllerTests.cs
│   ├── Program.cs                   # Alkalmazás belépési pont
```

```
│   ├── appsettings.json
│   ├── appsettings.Development.json
│   └── ThesisBackend.sln
├── ApiPlan.md                      # API tervezési dokumentáció
├── ApiPlan.yaml                    # OpenAPI specifikáció
├── README.md                       # Projekt README
└── Thesis_Draft.md                 # Szakdolgozat (ez a fájl)
```

C. Rövidítések jegyzéke

- **API:** Application Programming Interface
- **ASP.NET:** Active Server Pages .NET
- **CORS:** Cross-Origin Resource Sharing
- **CRUD:** Create, Read, Update, Delete
- **CSS:** Cascading Style Sheets
- **DTO:** Data Transfer Object
- **EF Core:** Entity Framework Core
- **HTML:** HyperText Markup Language
- **HTTP:** HyperText Transfer Protocol
- **HTTPS:** HyperText Transfer Protocol Secure
- **IoC:** Inversion of Control
- **JWT:** JSON Web Token
- **LINQ:** Language Integrated Query
- **MVC:** Model-View-Controller
- **ORM:** Object-Relational Mapping
- **REST:** Representational State Transfer
- **SPA:** Single Page Application
- **SQL:** Structured Query Language
- **TDD:** Test-Driven Development
- **UI:** User Interface
- **URL:** Uniform Resource Locator
- **XSS:** Cross-Site Scripting

Vége a szakdolgozatnak